RPA Architecture Specification

Release 0.1.0

Shanghai HappyFull Technologies

Jul 20, 2026

CHAPTER 1: INTRODUCTION

1 Overview **1**
- 1.1 The Challenge . . . 1
- 1.2 The RPA Approach . . . 2
- 1.3 Target Audience . . . 2

2 Key Features **3**
- 2.1 Extensible Isolation . . . 3
- 2.2 ISA-Independent Privilege Management . . . 3
- 2.3 Native Heterogeneous-ISA Support . . . 4
- 2.4 Security Domain Separation . . . 4
- 2.5 Unified Abstraction . . . 4

3 Terminology **7**
- 3.1 Core Concepts . . . 7
- 3.2 Address Concepts . . . 7
- 3.3 Modes . . . 8
- 3.4 Security Concepts . . . 8
- 3.5 ISA Concepts . . . 8
- 3.6 Hierarchy Concepts . . . 8

4 Domain Creation Example **9**
- 4.1 Scenario Overview . . . 9
- 4.2 Step 1: Firmware Creates Hypervisor Domain . . . 9
- 4.3 Step 2: Hypervisor Creates Guest OS Domain . . . 11
- 4.4 Step 3: Guest OS Creates User Application . . . 12
- 4.5 Summary Table . . . 13

5 System Call Example **15**
- 5.1 Scenario . . . 15
- 5.2 Setup . . . 15
- 5.3 Step 1: Application Prepares Arguments . . . 15
- 5.4 Step 2: Application Executes ascend . . . 15
- 5.5 Hardware Actions . . . 16
- 5.6 Step 3: Guest OS Handles System Call . . . 16
- 5.7 Step 4: Guest OS Returns to Application . . . 17
- 5.8 Hardware Actions for return . . . 17
- 5.9 Complete Timeline . . . 17
- 5.10 Overhead Analysis . . . 17
- 5.11 Hypervisor Interception (Optional) . . . 18

6 Interrupt Handling Examples **19**
6.1 Interrupt Delivery Overview . . . 19
6.2 Example 1: Locally Handled Interrupt . . . 19
6.3 Example 2: Propagated Interrupt . . . 21
6.4 Comparison . . . 22
6.5 Interrupt Controller Configuration . . . 23

7 DCB Memory Layout **25**
7.1 Three-Section Layout . . . 25
7.2 Alignment Requirements . . . 25
7.3 Memory Allocation . . . 25

8 DCB Fields **27**
8.1 RPA Spec Fields . . . 27
8.2 Field Semantics . . . 27
8.3 Access Permissions . . . 29

9 descend Primitive **31**
9.1 Instruction Format . . . 31
9.2 Pre-conditions . . . 31
9.3 Execution Steps . . . 31
9.4 Error Conditions . . . 32
9.5 Post-conditions . . . 32

10 ascend Primitive **33**
10.1 Instruction Format . . . 33
10.2 Execution Steps . . . 33
10.3 Trap Propagation . . . 34
10.4 Service Types . . . 34

11 return Primitive **35**
11.1 Instruction Format . . . 35
11.2 Execution Steps . . . 35
11.3 Use Cases . . . 35
11.4 Error Conditions . . . 36

12 Trap Mechanism **37**
12.1 Trap Sources . . . 37
12.2 Trap Handling . . . 37

13 Trap Propagation **39**
13.1 Propagation Chain . . . 39
13.2 Root Domain Behavior . . . 39

14 Interrupt Controller **41**
14.1 Interrupt Routing . . . 41
14.2 Interrupt Vector Table . . . 41
14.3 Interrupt Enable Mask . . . 41
14.4 Domain Isolation . . . 42

15 Interrupt Delivery **43**
15.1 Delivery Sequence . . . 43
15.2 Interrupt Propagation . . . 43

15.3 Example: Nested Virtualization . . . 44
15.4 Interrupt Priority . . . 44
15.5 Interrupt Latency . . . 45

16 Address Translation **47**
16.1 Translation Levels . . . 47
16.2 Translation Process . . . 47
16.3 Page Table Structure . . . 48
16.4 Translation Lookaside Buffer . . . 48

17 IPA Boundary **49**
17.1 IPA Region Definition . . . 49
17.2 Boundary Check . . . 49
17.3 Violation Handling . . . 50
17.4 Example: Application Memory Isolation . . . 50
17.5 Nested Virtualization Example . . . 50

18 INHERIT Mode **53**
18.1 Concept . . . 53
18.2 Performance Benefit . . . 53
18.3 Use Cases . . . 54
18.4 Security Considerations . . . 54
18.5 Mode Switching . . . 55

19 Memory Protection **57**
19.1 Protection Layers . . . 57
19.2 Permission Propagation . . . 57
19.3 Write-XOR-Execute (W^X) . . . 58
19.4 Memory Types . . . 58
19.5 Protection Example: Secure Enclave . . . 58

20 ISA Tag **61**
20.1 ISA Tag Field . . . 61
20.2 ISA Switch on Domain Transition . . . 61
20.3 ISA Boundary Rules . . . 62
20.4 Binary Translation Integration . . . 62
20.5 Implementation Requirements . . . 63

21 Context Management **65**
21.1 Context Structure . . . 65
21.2 Context Save on Domain Transition . . . 65
21.3 Context Restore . . . 66
21.4 Cross-ISA Calling Convention . . . 66
21.5 Context Size . . . 67

22 Privilege Model **69**
22.1 Traditional vs RPA . . . 69
22.2 Privilege Determination . . . 69
22.3 Privilege Inheritance . . . 70
22.4 Privilege Escalation . . . 70
22.5 Capability Model . . . 70

23 Security Overview **73**

23.1 Security Principles . . . 73
23.2 Threat Model . . . 73
23.3 Security Mechanisms . . . 73
23.4 Defense in Depth . . . 74
23.5 Security Configuration . . . 74

24 Security Groups **75**
24.1 Concept . . . 75
24.2 Security Group Definition . . . 75
24.3 Access Control Matrix . . . 76
24.4 Memory Access Check . . . 76
24.5 Example: Secure Enclave . . . 76
24.6 Management vs Access . . . 77

25 Memory Encryption **79**
25.1 Encryption Overview . . . 79
25.2 Per-Domain Encryption . . . 79
25.3 Encryption Modes . . . 80
25.4 Memory Bandwidth Impact . . . 80
25.5 Key Management . . . 81
25.6 Use Case: Confidential Computing . . . 81

26 Attestation **83**
26.1 Attestation Overview . . . 83
26.2 Attestation Components . . . 83
26.3 Attestation Keys . . . 84
26.4 Attestation Protocol . . . 84
26.5 Measurement Chain . . . 85

27 Confidential Computing **87**
27.1 Confidential Computing Model . . . 87
27.2 Threat Model . . . 87
27.3 RPA Confidential Computing Architecture . . . 87
27.4 Security Configuration . . . 88
27.5 VM Launch Protocol . . . 88
27.6 Use Cases . . . 89

28 System Diagram **91**
28.1 Domain Hierarchy . . . 91
28.2 Address Translation Flow . . . 91
28.3 Domain Transition . . . 92
28.4 Security Group Isolation . . . 93

29 Hardware-Software Interface **95**
29.1 DCB Memory Layout . . . 95
29.2 Instruction Encoding . . . 95
29.3 Control Registers . . . 96
29.4 Software Conventions . . . 96
29.5 Error Codes . . . 97

30 Boot Sequence **99**
30.1 Boot Stages . . . 99
30.2 Initial DCB Configuration . . . 99

30.3 Domain Creation During Boot . . . 100
30.4 Boot Example: Full System . . . 101
30.5 Secure Boot . . . 101

31 DCB Field Encoding **103**
31.1 Field Offsets . . . 103
31.2 Field Descriptions . . . 104
31.3 IPA Region Attributes . . . 105
31.4 Translation Control . . . 105

32 Pseudocode **107**
32.1 Reference Implementation . . . 107
32.2 descend Primitive . . . 107
32.3 ascend Primitive . . . 108
32.4 return Primitive . . . 109
32.5 IPA Boundary Check . . . 109
32.6 Security Group Check . . . 110

33 Custom Extensions **111**
33.1 Extension Points . . . 111
33.2 Custom DCB Fields . . . 111
33.3 Custom ISA Tags . . . 112
33.4 Custom Trap Types . . . 112
33.5 Custom Memory Attributes . . . 112
33.6 Extension Discovery . . . 113

34 References **115**
34.1 Related Architectures . . . 115
34.2 Normative References . . . 115
34.3 Informative References . . . 115
34.4 Glossary . . . 115

35 Legal Notice **117**
35.1 Copyright . . . 117
35.2 License . . . 117
35.3 Trademarks . . . 117
35.4 Disclaimers . . . 117
35.5 Implementations . . . 117
35.6 Contact . . . 118

36 Indices and tables **119**

Bibliography **121**

CHAPTER

ONE

OVERVIEW

Modern computing systems face two growing challenges: increasing virtualization nesting depth and expanding security domain requirements.

1.1 The Challenge

1.1.1 Virtualization Nesting Depth

Cloud infrastructure increasingly requires multiple layers of virtualization:

- Cloud providers run hypervisors on bare metal
- Tenants run nested hypervisors for isolation
- Container orchestrators add additional abstraction layers
- Sandbox environments require further isolation

Traditional architectures face limitations:

- x86 VT-x: 2-level nesting requires software emulation
- ARM: No native nested virtualization support
- RISC-V: H-extension limited to 2 levels

Each additional layer incurs 10-30% performance overhead when software emulation is required.

1.1.2 Security Domain Requirements

Confidential computing demands hardware-enforced isolation:

- Cloud tenants need protection from providers
- Multi-tenant systems require inter-tenant isolation
- Secure enclaves need nesting for complex workloads

Existing solutions lack nesting:

- Intel SGX/TDX: Single-level Trust Domains
- AMD SEV/SNP: No nested confidential VMs
- ARM CCA: Fixed-depth Realms

RPA addresses these challenges through a unified recursive model.

1.2 The RPA Approach

Recursive Privilege Architecture (RPA) decouples privilege management from instruction set design through:

1.2.1 Memory-Resident Configuration

Privilege configuration is stored in ordinary memory (Domain Control Block), not hardware registers:

- Unlimited nesting depth bounded by memory, not hardware
- Configuration is data-driven, not instruction-driven
- New privilege levels require no hardware changes

1.2.2 Unified Primitives

All privilege transitions use two symmetric primitives:

- **descend**: Enter a child domain
- **ascend**: Return to parent domain for service

These primitives replace:

- System calls (syscall/sysret)
- VM entry/exit
- TEE switches (SMC)
- Exception handling

1.2.3 ISA Independence

Privilege management is ISA-agnostic:

- Same architecture works on ARM, x86, RISC-V, IBM Z
- New ISAs need only define context format
- Heterogeneous-ISA domains can coexist

1.3 Target Audience

This specification is intended for:

- Hardware architects implementing RPA
- System software developers targeting RPA platforms
- Security architects designing confidential computing solutions
- Researchers studying privilege architecture

CHAPTER

TWO

KEY FEATURES

2.1 Extensible Isolation

RPA scales from microcontrollers to servers:

Table 1: Scalability Comparison

Platform	Configuration	Capability
MCU	INHERIT mode, 64-byte DCB	Lightweight kernel-user separation
Embedded	2-3 levels, minimal page tables	RTOS isolation, application sandboxing
Desktop	Full page tables, 4-8 levels	OS, hypervisor, sandbox support
Server	Unlimited levels, full features	Deep nesting, confidential computing

2.1.1 INHERIT Mode

When DCB pagetable field is 0, child shares parent's address space:

- Transition overhead: 5-10 cycles (vs 100-500 for full switch)
- No TLB flush required
- Suitable for lightweight isolation

2.1.2 Page Table Stacking

Each domain can have independent page table:

- $\text{VA} \rightarrow \text{IPA}_1 \rightarrow \text{IPA}_2 \rightarrow \ldots \rightarrow \text{PA}$
- Each level checks IPA boundaries
- Unlimited nesting depth

2.2 ISA-Independent Privilege Management

RPA overlays on any ISA with minimal modification:

Table 2: ISA Integration

ISA	Native Privilege	RPA Integration
ARM	EL0-EL3 (4 levels)	RPA replaces EL mechanism
x86	Ring 0-3 (4 levels)	RPA replaces Ring mechanism
RISC-V	M/S/U (3 modes)	RPA replaces mode mechanism
IBM Z	Supervisor/Problem (2 levels)	RPA extends nesting

New ISA design only needs to:

1. Define context save format
2. Implement two primitives (descend/ascend)
3. Define privilege level mapping

2.3 Native Heterogeneous-ISA Support

Domains with different ISAs can coexist:

- DCB includes isa_tag field
- Hardware performs context conversion on cross-ISA transitions
- Calling convention mapping is automatic

Use case: IBM-Arm collaboration for enterprise systems

2.4 Security Domain Separation

Management rights vs access rights:

Table 3: Rights Separation

Right Type	Description
Management	Parent configures child's ipa_regions, pagetable
Access	Parent cannot read child's private memory

This enables confidential computing:

- Cloud provider manages resources
- Cannot access tenant data
- Nested confidential VMs supported natively

2.5 Unified Abstraction

All privilege transitions use the same mechanism:

Table 4: Unified Transitions

Traditional Mechanism	RPA Equivalent	Description
syscall	ascend	User → kernel
VM entry	descend	Hypervisor → guest
VM exit	ascend	Guest → hypervisor
SMC	descend	Normal → secure world
Exception	implicit ascend	Fault propagation

This unification simplifies hardware design and enables composition of isolation mechanisms.

CHAPTER

THREE

TERMINOLOGY

3.1 Core Concepts

Domain
: A privilege domain is the fundamental isolation unit in RPA. Each domain has its own address space and privilege level. Domains are organized in a tree hierarchy.

Domain Control Block (DCB)
: Memory-resident data structure containing domain configuration. Stores page table base, IPA regions, exception vectors, and context.

descend
: Primitive operation to enter a child domain. Saves current context, loads child configuration, switches execution.

ascend
: Primitive operation to return to parent domain. Saves current context, switches to parent, jumps to trap handler.

return
: Primitive operation to resume child domain after service. Restores child context from DCB.

3.2 Address Concepts

Virtual Address (VA)
: Address used by software in current domain.

Intermediate Physical Address (IPA)
: Address after page table translation in child domain. Subject to IPA boundary checking before further translation.

Physical Address (PA)
: Final address in root domain's address space (actual DRAM).

IPA Regions
: Boundary constraints defining accessible address space for a domain. Set by parent, enforced by hardware.

3.3 Modes

INHERIT Mode
Child shares parent's address space (pagetable = 0). No address translation, only context save/restore.

Independent Mode
Child has its own page table (pagetable 0). Full VA → IPA translation with boundary checking.

3.4 Security Concepts

Security Group
Mechanism separating management rights from access rights. Binds domain to encryption keys and access policies.

Attestation
Report proving domain identity and integrity. Signed by hardware-derived keys.

Confidential Domain
Domain with memory encryption enabled. Parent cannot read child's data despite configuring resources.

3.5 ISA Concepts

isa_tag
DCB field identifying domain's instruction set. Enables heterogeneous-ISA support.

Context Conversion
Hardware process mapping registers between ISAs. Preserves calling convention semantics during cross-ISA transitions.

3.6 Hierarchy Concepts

Root Domain
Domain 0, the trust anchor. Highest privilege, owns all physical memory.

Child Domain
Domain created by parent via descend. Has restricted address space and lower privilege.

Relative Perspective
Each domain views itself as Domain 0. Children are Domain 1, 2, 3… Parent is Domain -1.

CHAPTER

FOUR

DOMAIN CREATION EXAMPLE

This chapter walks through a complete example: OS boot to application execution, showing how each layer creates child domains.

4.1 Scenario Overview

```
Domain 0: Firmware/Bootloader (Root)
│
├── Domain 1: Hypervisor
│   │
│   ├── Domain 2: Guest OS Kernel
│   │   │
│   │   └── Domain 3: User Application
│   │
│   └── Domain 2': Another Guest VM
│
└── Domain 1': System Management Mode
```

Each transition demonstrates DCB configuration for different subsystems.

4.2 Step 1: Firmware Creates Hypervisor Domain

Firmware (Domain 0) creates the hypervisor domain:

4.2.1 Memory Controller Configuration

```
// Firmware allocates memory for hypervisor
HYPERVISOR_BASE = 0x0010_0000    // 1 MB
HYPERVISOR_SIZE = 0x0100_0000    // 16 MB

// Create IPA regions for hypervisor
ipa_regions[0] = {
    base: 0x0000_0000,
    size: 0x0100_0000,    // 16 MB
    permissions: R|W|X
}

// Set DCB field
dcb.ipa_regions = &ipa_regions[0]
```

4.2.2 Interrupt Controller Configuration

```
// Request interrupt controller for hypervisor
irq_handle = sysop_irq_request(IRQ_POOL_HYPERVISOR)

// Store handle in DCB
dcb.interrupt_ctrl = irq_handle
```

4.2.3 Exception Handler Configuration

```
// Hypervisor will set its own trap handler after boot
// Firmware sets trap_vector = 0 initially
// (traps propagate to firmware if hypervisor not ready)
dcb.trap_vector = 0
```

4.2.4 Security Configuration

```
// No encryption for hypervisor (trusted)
dcb.security_group = 0

// ISA same as firmware
dcb.isa_tag = ISA_TAG_INHERIT
```

4.2.5 Page Table Configuration

```
// Hypervisor gets direct physical memory access
// Use INHERIT mode initially
dcb.pagetable = 0    // INHERIT mode
```

4.2.6 Entry Point

```
// Write hypervisor entry point
dcb.saved_lr = HYPERVISOR_ENTRY_POINT

// Size and alignment
dcb.ctrlblock_size = 8    // Minimum DCB
```

4.2.7 Execute descend

```
// Firmware executes
descend &dcb

// Hardware:
// 1. Assigns domain_id = 1
// 2. Saves firmware context
// 3. Switches to Domain 1
// 4. Jumps to HYPERVISOR_ENTRY_POINT
```

4.3 Step 2: Hypervisor Creates Guest OS Domain

Hypervisor (Domain 1) creates a guest VM:

4.3.1 Memory Configuration

```
// Allocate guest memory (may be encrypted)
GUEST_BASE = 0x1000_0000    // 256 MB
GUEST_SIZE = 0x1000_0000    // 256 MB

// IPA regions for guest
ipa_regions[0] = {
    base: 0x0000_0000,       // Guest sees 0-based addresses
    size: 0x1000_0000,
    permissions: R|W|X
}

dcb.ipa_regions = &ipa_regions[0]
```

4.3.2 Security Configuration

```
// Guest memory is encrypted (confidential computing)
sec_group = security_group_create(ENCRYPTION_ENABLED)
dcb.security_group = sec_group

// Now hypervisor cannot read guest memory
// (management rights only, no access rights)
```

4.3.3 Page Table Configuration

```
// Guest has independent address space
pagetable = create_guest_pagetable()
dcb.pagetable = pagetable

// Hardware will:
// 1. Translate guest VA → IPA
// 2. Check IPA against ipa_regions
// 3. Translate IPA → physical address
```

4.3.4 Interrupt Controller

```
// Virtual interrupt controller for guest
virq_handle = virq_controller_create()
dcb.interrupt_ctrl = virq_handle

// Hypervisor will intercept guest interrupts
```

4.3.5 Execute descend

```
descend &guest_dcb
```

4.4 Step 3: Guest OS Creates User Application

Guest OS kernel (Domain 2) creates user process:

4.4.1 Memory Configuration

```
// User process gets limited memory
ipa_regions[0] = {
    base: 0x0000_0000,
    size: 0x0100_0000,    // 1 MB for code/data
    permissions: R|X
}
ipa_regions[1] = {
    base: 0x0100_0000,
    size: 0x0010_0000,    // 64 KB for heap
    permissions: R|W
}
ipa_regions[2] = {
    base: 0x7FFF_0000,
    size: 0x0001_0000,    // 64 KB for stack
    permissions: R|W
}

dcb.ipa_regions = &ipa_regions[0]
```

4.4.2 Privilege Restriction

```
// User mode: no access to privileged instructions
// Trap handler in kernel
dcb.trap_vector = syscall_handler

// Independent address space
dcb.pagetable = user_pagetable
```

4.4.3 Execute descend

```
// Kernel creates user process
descend &user_dcb

// Application runs in Domain 3
// Can only access its assigned memory regions
```

4.5 Summary Table

Table 1: Domain Configuration Summary

Domain	Memory	Interrupt	Security	Page Table
0 (Firmware)	Full physical	Physical IRQ	None	Physical
1 (Hypervisor)	16 MB region	IRQ pool	None	INHERIT → Physical
2 (Guest OS)	256 MB encrypted	Virtual IRQ	Encrypted	Guest VA → IPA → PA
3 (Application)	1 MB code + heap	None (trap to OS)	Inherit guest	User VA → Guest IPA

CHAPTER

FIVE

SYSTEM CALL EXAMPLE

This chapter demonstrates how an application executes a system call through the ascend primitive.

5.1 Scenario

Application (Domain 3) requests file I/O from Guest OS (Domain 2).

5.2 Setup

```
Domain 3: User Application
Domain 2: Guest OS Kernel
Domain 1: Hypervisor (optional interception)
Domain 0: Firmware
```

Application wants to read from a file:

```
// Application code
int fd = open("/data/file.txt", O_RDONLY);
char buffer[1024];
read(fd, buffer, 1024);
```

5.3 Step 1: Application Prepares Arguments

```
// Application prepares system call
R0 = SYS_read        // System call number
R1 = fd              // File descriptor
R2 = buffer          // Buffer address
R3 = 1024            // Count
```

5.4 Step 2: Application Executes ascend

```
ascend SYS_read
```

5.5 Hardware Actions

5.5.1 Context Save

```
// Hardware saves current context to Domain 3 DCB
dcb3.saved_pc = PC + 4
dcb3.saved_sp = SP
dcb3.saved_psr = PSR

// Arguments preserved in registers (ISA-specific)
```

5.5.2 Domain Switch

```
// Hardware switches to parent
current_domain = domain_2
```

5.5.3 Trap Vector Resolution

```
// Hardware checks Domain 2 trap_vector
if dcb2.trap_vector != 0:
    PC = dcb2.trap_vector    // Jump to syscall handler
else:
    // Propagate to Domain 1 (and so on)
```

5.6 Step 3: Guest OS Handles System Call

```
// Guest OS syscall handler
syscall_handler:
    // Save user registers
    push {r4-r11}

    // Dispatch based on R0
    cmp R0, #SYS_read
    beq handle_read
    cmp R0, #SYS_write
    beq handle_write
    ...

handle_read:
    // Validate fd and buffer address
    // Check buffer is in user's ipa_regions
    // Perform read operation
    // Store result in R0
    // Return to user
```

5.7 Step 4: Guest OS Returns to Application

```
// Kernel prepares return
R0 = bytes_read    // Return value

// Execute return
return
```

5.8 Hardware Actions for return

```
// Hardware restores Domain 3 context
PC = dcb3.saved_pc
SP = dcb3.saved_sp
PSR = dcb3.saved_psr

// Application continues with return value in R0
```

5.9 Complete Timeline

```
Time | Domain | Action
─────────────────────────────────────────────
 T0  |    3   | Application executes ascend R0
 T1  |   HW   | Save Domain 3 context to DCB3
 T2  |   HW   | Switch current_domain → 2
 T3  |   HW   | Load PC = dcb2.trap_vector
 T4  |    2   | Guest OS syscall handler runs
 T5  |    2   | Handle read request
 T6  |    2   | Execute return
 T7  |   HW   | Restore Domain 3 context
 T8  |    3   | Application continues
```

5.10 Overhead Analysis

Table 1: System Call Overhead

Operation	Cycles	Notes
Context save	5-10	Register store to DCB
Domain switch	1-2	Update current_domain pointer
Trap dispatch	2-5	Load trap_vector, jump
Total (ascend)	8-17	Ascend overhead only

Compare to traditional syscall:

- x86 syscall/sysret: 100-500 cycles
- ARM SVC: 50-200 cycles
- RPA ascend: 8-17 cycles

5.11 Hypervisor Interception (Optional)

If hypervisor needs to intercept guest system calls:

```
// Guest OS sets trap_vector = 0
dcb2.trap_vector = 0

// When application ascends:
// 1. Hardware checks dcb2.trap_vector = 0
// 2. Propagates to Domain 1
// 3. Hypervisor's trap handler executes
```

This enables:

- System call filtering
- Virtual device emulation
- Security policy enforcement

CHAPTER

SIX

INTERRUPT HANDLING EXAMPLES

This chapter demonstrates two interrupt scenarios: one handled locally, one propagated to parent.

6.1 Interrupt Delivery Overview

When an interrupt occurs:

1. Hardware identifies target domain
2. If domain has interrupt handler, deliver
3. If not, propagate to parent domain
4. Repeat until handled

6.2 Example 1: Locally Handled Interrupt

Scenario: Timer interrupt for Guest OS scheduler.

6.2.1 Setup

```
Domain 2: Guest OS
- Has timer configured
- Has interrupt handler registered

Domain 3: User Application (running)
- No timer access
- Will be preempted
```

6.2.2 Interrupt Configuration

```
// Guest OS registered timer interrupt
irq_timer = irq_request(IRQ_TYPE_TIMER)
dcb2.interrupt_ctrl = irq_controller

// Registered handler address
timer_handler = 0x8000_1000
```

6.2.3 Interrupt Arrival

```
// Hardware timer fires
// Interrupt controller signals Domain 2
```

6.2.4 Hardware Actions

```
// Domain 3 is running, Domain 2 owns the interrupt

1. Save Domain 3 context (if different owner)
   dcb3.saved_pc = PC
   dcb3.saved_sp = SP
   dcb3.saved_psr = PSR

2. Switch to Domain 2
   current_domain = domain_2

3. Check if Domain 2 handles this interrupt
   irq_owner = interrupt_get_owner(irq_timer)
   if irq_owner == domain_2:
       // Deliver locally

4. Jump to interrupt handler
   PC = dcb2.interrupt_handler
```

6.2.5 Guest OS Handler

```
timer_handler:
    // Save current context
    push {r0-r12, lr}

    // Handle timer: scheduler tick
    bl scheduler_tick

    // Restore and return to Domain 3
    pop {r0-r12, lr}
    return
```

6.2.6 Timeline

```
Time | Domain | Action
─────────────────────────────────────────────
 T0  |    3   | Application running
 T1  |    HW  | Timer interrupt arrives
 T2  |    HW  | Save Domain 3 context
 T3  |    HW  | Switch to Domain 2
 T4  |    HW  | Jump to timer_handler
 T5  |    2   | Guest OS scheduler runs
 T6  |    2   | Execute return
```

(continues on next page)

(continued from previous page)

```
T7  |   HW   | Restore Domain 3 context
T8  |   3    | Application resumes
```

6.3 Example 2: Propagated Interrupt

Scenario: Network interrupt while User Application running.

6.3.1 Setup

```
Domain 2: Guest OS
- Has network driver
- trap_vector = 0 (propagate unhandled traps)

Domain 3: User Application
- No interrupt handlers
- Running computation
```

6.3.2 Interrupt Configuration

```
// Network interrupt owned by Hypervisor (Domain 1)
// Guest OS does not own network hardware

dcb2.interrupt_ctrl = virtual_irq_only
dcb2.trap_vector = 0    // Propagate unhandled
```

6.3.3 Interrupt Arrival

```
// Network interrupt arrives
// Target: Domain 1 (Hypervisor)
```

6.3.4 Hardware Actions

```
// Domain 3 running, interrupt for Domain 1

1. Implicit ascend from Domain 3
   // Domain 3 has no handler, trap to Domain 2
   dcb3.saved_pc = PC
   current_domain = domain_2

2. Check Domain 2 trap_vector
   if dcb2.trap_vector == 0:
       // Propagate to Domain 1

3. Implicit ascend to Domain 1
   dcb2.saved_pc = PC
   current_domain = domain_1
```

(continues on next page)

(continued from previous page)

```
4. Check Domain 1 interrupt ownership
   if interrupt_owner == domain_1:
       PC = dcb1.network_handler
```

6.3.5 Hypervisor Handler

```
network_handler:
    // Read network packet
    // Determine target guest
    // Inject virtual interrupt to Domain 2
    virq_inject(domain_2, VIRQ_NETWORK)

    // Return down the chain
    return    // To Domain 2
    return    // To Domain 3
```

6.3.6 Timeline

Time	Domain	Action
T0	3	Application running
T1	HW	Network interrupt (for Domain 1)
T2	HW	Implicit ascend: 3 → 2
T3	HW	trap_vector = 0, propagate
T4	HW	Implicit ascend: 2 → 1
T5	HW	Jump to network_handler
T6	1	Hypervisor handles packet
T7	1	return (to Domain 2)
T8	HW	Restore Domain 2 context
T9	2	return (to Domain 3)
T10	HW	Restore Domain 3 context
T11	3	Application resumes

6.4 Comparison

Table 1: Interrupt Handling Comparison

Aspect	Local Handling	Propagated Handling
Handler location	Same domain as interrupt owner	Ancestor domain
Context switches	1 (current → handler)	N (propagate through N levels)
Latency	Lower	Higher (proportional to levels)
Use case	Guest OS timer, device	Shared hardware, security

6.5 Interrupt Controller Configuration

6.5.1 Per-Domain Configuration

```
struct interrupt_config {
    irq_mask;           // Which IRQs this domain handles
    irq_handler;        // Handler address
    propagate_mask;     // IRQs to always propagate
}

// Domain 2 (Guest OS)
interrupt_config.irq_mask = {TIMER, UART}
interrupt_config.propagate_mask = {NETWORK, DISK}

// Domain 1 (Hypervisor)
interrupt_config.irq_mask = {ALL}
```

CHAPTER

SEVEN

DCB MEMORY LAYOUT

The Domain Control Block resides in ordinary memory, aligned to 8-word boundaries.

7.1 Three-Section Layout

```
┌─────────────────────────────────────────┐
│ RPA Spec Fields (8 words, fixed)        │ Offset 0x00-0x1F
│ Architecture-defined, cross-platform    │
├─────────────────────────────────────────┤
│ RPA Impdef Fields (variable)            │ Offset 0x20+
│ Implementation-defined extensions       │
├─────────────────────────────────────────┤
│ ISA Context Fields (variable)           │
│ ISA-specific context save area          │
└─────────────────────────────────────────┘
```

7.2 Alignment Requirements

- DCB address must be 8-word aligned (32-byte boundary for 32-bit systems)
- Minimum size: 8 words (RPA Spec Fields only)
- ctrlblock_size field specifies total size in words

7.3 Memory Allocation

The parent domain allocates DCB memory:

```
// Allocate aligned memory
dcb_addr = aligned_alloc(32, dcb_size * 4);

// Initialize to zero
memset(dcb_addr, 0, dcb_size * 4);

// Set size field
dcb_addr->ctrlblock_size = dcb_size;
```

The child domain cannot access its own DCB through direct memory access.

CHAPTER

EIGHT

DCB FIELDS

8.1 RPA Spec Fields

Table 1: RPA Spec Field Encoding

Offset	Field	Set By	Description
0x00	ctrlblock_size	Parent	Control block size in words (min 8)
0x04	domain_id	System	Unique domain identifier
0x08	trap_vector	Child	Trap handler entry (0 = propagate to parent)
0x0C	interrupt_ctrl	System	Interrupt controller handle
0x10	ipa_regions	Parent	IPA boundary table address
0x14	pagetable	Child	Page table base (0 = INHERIT mode)
0x18	child_block	Parent	Child DCB address for return
0x1C	security_group	System	Security group handle
0x20	isa_tag	Parent	ISA identifier

8.2 Field Semantics

8.2.1 ctrlblock_size

Size of the entire DCB in words. Used by hardware to validate DCB during descend.

- Minimum: 8 words (RPA Spec Fields only)
- Alignment: 8 words
- Must be set by parent before descend

8.2.2 domain_id

System-assigned unique identifier for the domain.

- Assigned by hardware on first descend
- Used for DMA access control
- Used for security group binding

8.2.3 trap_vector

Entry point for trap handling.

- Set by child domain after initialization
- Value 0: Traps propagate to parent
- Non-zero: Hardware jumps to this address on trap

8.2.4 interrupt_ctrl

Handle for the domain's interrupt controller.

- Allocated by system via sysop instruction
- Determines which interrupts this domain handles

8.2.5 ipa_regions

Address of IPA boundary table.

- Set by parent, read-only for child
- Defines accessible address space
- Hardware checks all translations against these bounds

8.2.6 pagetable

Page table base address for VA→IPA translation.

- Set by child domain
- Value 0: INHERIT mode (share parent address space)
- Non-zero: Independent address translation

8.2.7 child_block

Address of child domain's DCB.

- Maintained by parent
- Used by return instruction to resume child
- Cleared when child is destroyed

8.2.8 security_group

Security group handle for memory isolation.

- Allocated by system
- Enables memory encryption
- Separates management rights from access rights

8.2.9 isa_tag

ISA identifier for heterogeneous-ISA support.

Table 2: ISA Tag Values

Value	ISA
0x0000	INHERIT (same as parent)
0x0001	ARM/AArch32
0x0002	RISC-V
0x0003	x86-64
0x0004	IBM Z

8.3 Access Permissions

Table 3: DCB Field Access

Field	Parent	Child	Hardware
ctrlblock_size	R/W	•	R
domain_id	•	•	R/W
trap_vector	•	R/W	R
ipa_regions	R/W	R	R
pagetable	•	R/W	R
child_block	R/W	•	R
security_group	•	•	R/W
isa_tag	R/W	•	R

CHAPTER

NINE

DESCEND PRIMITIVE

The descend primitive enters a child domain for execution.

9.1 Instruction Format

```
descend <dcb_address>

dcb_address: Register or immediate containing DCB address
```

9.2 Pre-conditions

Parent domain must:

1. Allocate and initialize DCB
2. Set ctrlblock_size
3. Configure ipa_regions
4. Set entry point in saved_lr field

9.3 Execution Steps

```
descend R0 (R0 = child_dcb_address):

Step 1: DCB Validation
──────────────────────
size = memory[R0 + 0x00]

if R0 % 32 != 0:
    fault(DCB_ALIGNMENT_ERROR)

if size < 8:
    fault(DCB_SIZE_ERROR)

Step 2: Domain ID Assignment
────────────────────────────
domain_id = next_domain_id++
memory[R0 + 0x04] = domain_id
```

(continues on next page)

(continued from previous page)

```
Step 3: Parent Context Save
_____________________________
parent_dcb.saved_pc = PC + 4
parent_dcb.saved_sp = SP
parent_dcb.saved_psr = PSR

Step 4: Link Child
____________________
parent_dcb.child_block = R0

Step 5: Domain Switch
_______________________
current_domain = child_domain

Step 6: Load Child Entry
__________________________
PC = memory[R0 + saved_lr_offset]
```

9.4 Error Conditions

Table 1: descend Errors

Error	Cause
DCB_ALIGNMENT	DCB address not 8-word aligned
DCB_SIZE	ctrlblock_size < 8
IPA_FAULT	DCB address not in accessible memory
PERMISSION	Insufficient privilege for descend

9.5 Post-conditions

After successful descend:

- current_domain points to child
- PC points to child entry point
- Parent's context saved in parent DCB
- Child's domain_id assigned

CHAPTER

TEN

ASCEND PRIMITIVE

The ascend primitive returns to parent domain to request service.

10.1 Instruction Format

```
ascend <service_type>

service_type: Register indicating requested service
```

10.2 Execution Steps

```
ascend R0 (R0 = service_type):

Step 1: Context Save
————————————————————
current_dcb.saved_pc = PC + 4
current_dcb.saved_sp = SP
current_dcb.saved_psr = PSR

// Pass service type in designated register
parent_context.arg_reg = R0

Step 2: Domain Switch
————————————————————
current_domain = current_domain.parent

Step 3: Trap Vector Resolution
——————————————————————————
while current_dcb.trap_vector == 0:
    if current_domain.is_root:
        fault(UNHANDLED_TRAP)
    current_domain = current_domain.parent

Step 4: Jump to Handler
———————————————————
PC = current_dcb.trap_vector
```

10.3 Trap Propagation

When trap_vector is 0, the trap propagates up the hierarchy:

```
Domain 3: ascend (trap_vector = 0)
    |
    Propagate
Domain 2: (trap_vector = 0)
    |
    Propagate
Domain 1: (trap_vector = handler)
    |
    Jump to handler
```

This enables:

- Default fault handling at root
- Selective interception at intermediate levels
- Security policy enforcement

10.4 Service Types

Table 1: Standard Service Types

Value	Service
0x00	System call
0x01	Page fault
0x02	I/O request
0x03	Security violation
0x04-0xFF	Implementation-defined

CHAPTER

ELEVEN

RETURN PRIMITIVE

The return primitive resumes child domain after parent completes service.

11.1 Instruction Format

```
return
```

11.2 Execution Steps

```
return:

Step 1: Locate Child DCB
————————————————————————
child_dcb = current_dcb.child_block

if child_dcb == 0:
    fault(NO_CHILD_DOMAIN)

Step 2: Domain Switch
————————————————————
current_domain = child_domain

Step 3: Context Restore
———————————————————————
PC = child_dcb.saved_pc
SP = child_dcb.saved_sp
PSR = child_dcb.saved_psr

// Return value in designated register
R0 = child_dcb.return_value
```

11.3 Use Cases

11.3.1 System Call Return

```
// In parent (kernel)
// After handling system call
```

(continues on next page)

(continued from previous page)

```
R0 = return_value    // Set return value
return               // Resume child
```

11.3.2 Nested Virtualization

```
// Hypervisor returns to guest after VM exit handling
return

// Hardware restores guest context from child DCB
```

11.4 Error Conditions

Table 1: return Errors

Error	Cause
NO_CHILD	child_block is 0
CHILD_NOT_SUSPEN	Child is not in suspended state

CHAPTER

TWELVE

TRAP MECHANISM

Traps are synchronous exceptions that cause implicit ascend to parent domain.

12.1 Trap Sources

Table 1: Trap Sources

Source	Description
System call	Explicit ascend by software
Page fault	Address translation failure
Privilege violation	Privileged instruction in user mode
IPA boundary violation	Access outside permitted regions
Alignment fault	Misaligned memory access

12.2 Trap Handling

```
On trap:
1. Hardware saves context to current DCB
2. Hardware performs implicit ascend
3. Parent's trap_handler executes
4. Handler either:
   - Resolves and returns
   - Propagates to its parent
   - Terminates domain
```

CHAPTER

THIRTEEN

TRAP PROPAGATION

When a domain's trap_vector is 0, traps propagate to parent.

13.1 Propagation Chain

```
Domain N: trap occurs
    │
    │ trap_vector = 0?
    ├─ No → Execute handler
    │
    └─ Yes → Propagate to Domain N-1
             │
             │ trap_vector = 0?
             ├─ No → Execute handler
             │
             └─ Yes → Continue propagation
```

13.2 Root Domain Behavior

If trap reaches root domain with trap_vector = 0:

```
// Root must handle all unhandled traps
if root_dcb.trap_vector == 0:
    // System error
    halt()
```

CHAPTER

FOURTEEN

INTERRUPT CONTROLLER

RPA domains interact with interrupt controllers through domain-relative interrupt routing.

14.1 Interrupt Routing

Each domain can receive interrupts from:

Table 1: Interrupt Sources

Source	Description
Device interrupts	From devices assigned to the domain
Timer interrupts	Per-domain virtual timers
Inter-processor interrupts	From other domains or processors
Software interrupts	Self-generated via IPI mechanism

14.2 Interrupt Vector Table

Each DCB contains an interrupt vector table base:

```
DCB Field: interrupt_vector
──────────────────────────
Address of interrupt vector table

Vector Table Entry (per interrupt):
──────────────────────────────────
┌──────────────────────────────────┐
│ handler_address (64-bit)         │
├──────────────────────────────────┤
│ interrupt_attributes (32-bit)    │
└──────────────────────────────────┘
```

14.3 Interrupt Enable Mask

DCB contains an interrupt enable mask:

```
DCB Field: interrupt_mask
─────────────────────────
Bit N = 1: Interrupt N enabled
```

(continues on next page)

(continued from previous page)

```
Bit N = 0: Interrupt N disabled

Hardware checks mask before delivery:
if interrupt_mask[interrupt_id] == 0:
    // Interrupt pending but not delivered
    pending_interrupts |= (1 << interrupt_id)
```

14.4 Domain Isolation

Interrupts are isolated between domains:

```
Domain A interrupt does NOT affect Domain B:
┌──────────────────────────────────────────┐
│ Domain A: Timer interrupt → Handler A    │
├──────────────────────────────────────────┤
│ Domain B: No interrupt visible           │
└──────────────────────────────────────────┘

This isolation is enforced by:
1. Per-domain interrupt_mask
2. Per-domain interrupt_vector
3. Hardware routing based on domain assignment
```

CHAPTER

FIFTEEN

INTERRUPT DELIVERY

Interrupt delivery in RPA follows a domain-relative model with automatic propagation support.

15.1 Delivery Sequence

When an interrupt occurs:

```
Step 1: Identify Target Domain
──────────────────────────────
target_domain = interrupt_routing[interrupt_id]

Step 2: Check Interrupt Mask
────────────────────────────
if target_dcb.interrupt_mask[interrupt_id] == 0:
    pending_interrupts |= (1 << interrupt_id)
    return  // Interrupt not delivered

Step 3: Check Handler
─────────────────────
if target_dcb.interrupt_vector == 0:
    // No handler: propagate to parent
    propagate_interrupt(target_domain, interrupt_id)
    return

Step 4: Deliver to Handler
──────────────────────────
// Save context
target_dcb.saved_pc = PC
target_dcb.saved_sp = SP
target_dcb.saved_psr = PSR

// Jump to handler
PC = target_dcb.interrupt_vector + (interrupt_id * 16)
```

15.2 Interrupt Propagation

If a domain has no interrupt handler (interrupt_vector = 0):

```
propagate_interrupt(domain, interrupt_id):

while domain.dcb.interrupt_vector == 0:
    if domain.is_root:
        // Unhandled interrupt at root
        fault(UNHANDLED_INTERRUPT)
        return

    domain = domain.parent

// Deliver to ancestor with handler
deliver_to_handler(domain, interrupt_id)
```

15.3 Example: Nested Virtualization

```
Interrupt delivery in nested VM:

┌─────────────────────────────────────────┐
│ Domain 0: Root (Hypervisor)             │
│   interrupt_vector = 0x8000             │
└─────────────────────────────────────────┘
                  │ Propagate (no handler)
┌─────────────────────────────────────────┐
│ Domain 1: Guest Hypervisor              │
│   interrupt_vector = 0                  │
└─────────────────────────────────────────┘
                  │ Propagate (no handler)
┌─────────────────────────────────────────┐
│ Domain 2: Guest OS                      │
│   interrupt_vector = 0                  │
└─────────────────────────────────────────┘
                  │ Interrupt occurs
┌─────────────────────────────────────────┐
│ Domain 3: Guest Application             │
│   (Running, interrupted)                │
└─────────────────────────────────────────┘

Result: Root hypervisor handles all device interrupts
Guest hypervisor can intercept by setting interrupt_vector
```

15.4 Interrupt Priority

Interrupts have configurable priority levels:

```
DCB Field: interrupt_priority[32]
_____________________________________

Priority for each interrupt (0 = highest)

Hardware behavior:
if pending_interrupts != 0:
    highest = find_highest_priority(pending_interrupts)
    deliver(highest)
```

15.5 Interrupt Latency

RPA minimizes interrupt latency through:

Table 1: Latency Factors

Factor	Impact
No privilege escalation	Direct delivery to domain
Single context switch	Hardware saves/restores context
Maskable propagation	Optional interception at any level

CHAPTER

SIXTEEN

ADDRESS TRANSLATION

RPA extends traditional address translation with multi-level translation through the domain hierarchy.

16.1 Translation Levels

RPA supports configurable translation levels:

```
Level 0: Virtual Address (VA)
──────────────────────────────
Address generated by executing code

Level 1: Intermediate Physical Address (IPA)
─────────────────────────────────────────────
Address after first-level translation

Level 2+: Physical Address (PA)
────────────────────────────────
Final address after all translations
```

16.2 Translation Process

```
Memory access translation:

Step 1: VA → IPA Translation
─────────────────────────────
ipa = translate_va_to_ipa(va, current_dcb.page_table)

Step 2: IPA Boundary Check
──────────────────────────
if ipa not in current_dcb.ipa_regions:
    fault(IPA_BOUNDARY_VIOLATION)

Step 3: IPA → PA Translation (if INHERIT mode disabled)
────────────────────────────────────────────────────────
if current_dcb.translation_mode != INHERIT:
    pa = translate_ipa_to_pa(ipa, parent_dcb.page_table)
else:
    pa = ipa  // Direct mapping in INHERIT mode
```

16.3 Page Table Structure

Each DCB contains page table base:

```
DCB Fields:
─────────────────────────────────────────────────────
page_table_root    : Base address of page table
translation_control: Translation configuration

Page Table Entry Format:
─────────────────────────────────────────────────────
┌──────────────────────────────────────────────────┐
│ [63:48] Reserved                                 │
├──────────────────────────────────────────────────┤
│ [47:12] Next Level / Physical Frame Number       │
├──────────────────────────────────────────────────┤
│ [11:0]  Permissions and Attributes               │
│   - Bit 0: Valid                                 │
│   - Bit 1: Readable                              │
│   - Bit 2: Writable                              │
│   - Bit 3: Executable                            │
│   - Bit 4: User accessible                       │
│   - Bits 5-7: Memory attributes                  │
└──────────────────────────────────────────────────┘
```

16.4 Translation Lookaside Buffer

RPA TLB entries include domain context:

```
TLB Entry:
─────────────────────────────────────────────────────
┌──────────────┬──────────┬──────────┬──────────────┐
│ domain_id    │ VA tag   │ IPA/PA   │ Permissions  │
└──────────────┴──────────┴──────────┴──────────────┘

TLB lookup:
entry = TLB.lookup(va, current_domain.id)
if entry.valid:
    return entry.physical_address

TLB flush on domain switch:
if new_domain.id != old_domain.id:
    // Optional: selective flush
    TLB.flush_domain(old_domain.id)
```

CHAPTER

SEVENTEEN

IPA BOUNDARY

IPA (Intermediate Physical Address) boundaries define the memory regions accessible to each domain.

17.1 IPA Region Definition

Each DCB contains a list of permitted IPA regions:

```
DCB Field: ipa_regions[]
_________________________

Array of IPA region descriptors

Region Descriptor:
_____________________________________________________
| base_ipa   : Starting IPA (64-bit)                |
|___________________________________________________|
| limit_ipa  : Ending IPA (64-bit, inclusive)       |
|___________________________________________________|
| attributes : Access permissions                   |
|   - Readable (R)                                  |
|   - Writable (W)                                  |
|   - Executable (X)                                |
|   - Device memory (D)                             |
|___________________________________________________|
```

17.2 Boundary Check

Hardware checks IPA boundaries on every memory access:

```
check_ipa_boundary(ipa, access_type):

for region in current_dcb.ipa_regions:
    if ipa >= region.base_ipa and ipa <= region.limit_ipa:
        // Check access permissions
        if access_type == READ and not region.R:
            fault(PERMISSION_DENIED)
        if access_type == WRITE and not region.W:
            fault(PERMISSION_DENIED)
        if access_type == EXECUTE and not region.X:
            fault(PERMISSION_DENIED)
```

(continues on next page)

(continued from previous page)

```
        return ALLOWED

// IPA not in any permitted region
fault(IPA_BOUNDARY_VIOLATION)
```

17.3 Violation Handling

IPA boundary violations trigger traps:

```
On IPA_BOUNDARY_VIOLATION:
1. Hardware saves context to current DCB
2. Trap propagates to parent domain
3. Parent handler decides:
   - Map new region (extend access)
   - Terminate domain (security violation)
   - Emulate access (device I/O)
```

17.4 Example: Application Memory Isolation

```
Application DCB:
┌─────────────────────────────────────────────────┐
│ ipa_regions[0]:                                 │
│   base  = 0x00010000                            │
│   limit = 0x0001FFFF  (64KB code)               │
│   attr  = R-X                                   │
├─────────────────────────────────────────────────┤
│ ipa_regions[1]:                                 │
│   base  = 0x00020000                            │
│   limit = 0x0002FFFF  (64KB data)               │
│   attr  = RW-                                   │
└─────────────────────────────────────────────────┘

Attempt to access IPA 0x00030000:
→ IPA_BOUNDARY_VIOLATION
→ Trap to parent (OS)
→ OS may extend region or terminate application
```

17.5 Nested Virtualization Example

```
Guest VM IPA regions (set by hypervisor):

Guest DCB:
┌─────────────────────────────────────────────────┐
│ ipa_regions[0]:                                 │
│   base  = 0x40000000                            │
│   limit = 0x4FFFFFFF  (256MB guest physical)    │
│   attr  = RWX                                   │
```

(continues on next page)

(continued from previous page)

```
└──────────────────────────────────────────────────┘


Hypervisor translates guest IPA to host PA:
- Guest IPA 0x40000000 → Host PA 0x80000000
- Guest IPA 0x4FFFFFFF → Host PA 0x8FFFFFFF

Guest cannot access outside its assigned region.
```

CHAPTER

EIGHTEEN

INHERIT MODE

INHERIT mode enables efficient address space sharing between parent and child domains.

18.1 Concept

INHERIT mode allows a child domain to use the parent's page tables:

```
DCB Field: translation_mode
_____________________________

INHERIT  (0): Use parent's page tables
OWN      (1): Use own page tables

When INHERIT is set:
- VA → IPA translation uses parent's page tables
- No separate page table for child
- Address space fully shared with parent
```

18.2 Performance Benefit

INHERIT mode eliminates double translation overhead:

```
Traditional (no INHERIT):
VA → IPA (child page table) → PA (parent page table)
Cost: 2 page table walks

INHERIT mode:
VA → PA (parent page table directly)
Cost: 1 page table walk

System call overhead comparison:
_____________________________________________
| Traditional: 100-500 cycles               |
| INHERIT:     1-10 cycles                  |
|___________________________________________|
```

18.3 Use Cases

18.3.1 System Calls

```
Application domain (INHERIT mode):
- Shares OS page table
- ascend to OS is nearly free
- No TLB flush on domain switch

Timeline:
1. Application executes in child domain
2. ascend for system call
3. OS handler runs with same address space
4. return to application
- No page table switch
- No TLB invalidation
```

18.3.2 Nested Virtualization

```
Guest hypervisor with INHERIT:
- Can share root hypervisor's address space
- VM exit handling is fast
- Nested virtualization overhead reduced

Domain 0: Root hypervisor
  └─ Domain 1: Guest hypervisor (INHERIT from Domain 0)
     └─ Domain 2: Guest OS
        └─ Domain 3: Guest application
```

18.4 Security Considerations

INHERIT mode does NOT bypass IPA boundaries:

```
Even with INHERIT mode:
1. Child's IPA regions still enforced
2. Child cannot access memory outside its regions
3. Parent can restrict child's view

Example:
┌─────────────────────────────────────────────────┐
│ Parent (OS) has full memory access              │
│ Child (Application) with INHERIT:               │
│   - Uses parent's page tables                   │
│   - But IPA regions limit to:                   │
│     - Code: 0x10000-0x1FFFF                     │
│     - Data: 0x20000-0x2FFFF                     │
│   - Cannot access other memory                  │
└─────────────────────────────────────────────────┘
```

18.5 Mode Switching

Translation mode can be changed by parent:

```
// Parent switches child from OWN to INHERIT
child_dcb.translation_mode = INHERIT
child_dcb.page_table_root = 0  // Not used

// Must flush TLB
TLB.flush_domain(child_domain.id)
```

CHAPTER

NINETEEN

MEMORY PROTECTION

RPA provides multi-layer memory protection through IPA boundaries, page permissions, and domain isolation.

19.1 Protection Layers

```
Layer 1: Page Table Permissions
─────────────────────────────────
Per-page R/W/X attributes

Layer 2: IPA Boundary Checks
──────────────────────────────
Region-level access control

Layer 3: Domain Isolation
───────────────────────────
Complete address space separation
```

19.2 Permission Propagation

Child domains cannot exceed parent permissions:

```
Permission intersection rule:

child_effective_perm = child_page_perm  parent_page_perm

Example:
┌──────────────────────────────────────────────┐
│ Parent page: RW-                             │
│ Child page: RWX                              │
│ Effective:   RW-  (execute denied)           │
└──────────────────────────────────────────────┘

Hardware enforces this automatically:
- Child page table may claim RWX
- Parent page table limits to RW-
- Actual access is intersection
```

19.3 Write-XOR-Execute (W^X)

RPA supports W^X enforcement:

```
W^X rule: A page cannot be both writable and executable

Implementation:
if page.W and page.X:
    fault(W_X_VIOLATION)

This can be enforced at:
- Page table level (hardware check)
- IPA region level (region attributes)
- Domain level (security policy)
```

19.4 Memory Types

RPA distinguishes memory types:

Table 1: Memory Types

Type	Description
Normal	Cacheable, reorderable memory
Device	Non-cacheable, side-effect memory
Strongly-ordered	Non-cacheable, non-reorderable

```
Memory type in IPA region:
region.attributes.device = 1  // Device memory

Hardware behavior:
- Device memory: No speculative reads
- Device memory: No write combining
- Device memory: Strict ordering
```

19.5 Protection Example: Secure Enclave

```
Secure enclave memory layout:

┌─────────────────────────────────────────────┐
│ Domain: Secure Enclave                      │
│                                             │
│ IPA Region 0 (Code):                        │
│   base  = 0x00000000                        │
│   limit = 0x0000FFFF                        │
│   attr  = R-X (read-only, executable)       │
│                                             │
│ IPA Region 1 (Data):                        │
│   base  = 0x00010000                        │
```

(continues on next page)

(continued from previous page)

```
  limit = 0x0001FFFF                                  
  attr  = RW- (read-write, no execute)                
                                                      
 IPA Region 2 (Shared buffer):                        
  base  = 0x00020000                                  
  limit = 0x0002FFFF                                  
  attr  = RW- (shared with parent)                    
                                                      
 No other regions → Cannot access outside memory      
└─────────────────────────────────────────────────────┘


Parent (OS) can:
- Read/write shared buffer
- Cannot read enclave private memory
- If security groups enabled, OS cannot even access
```

CHAPTER

TWENTY

ISA TAG

RPA supports heterogeneous-ISA systems through ISA tagging of domains.

20.1 ISA Tag Field

Each DCB contains an ISA identifier:

```
DCB Field: isa_tag
──────────────────
Identifies the instruction set architecture

Standard ISA Tags:
┌──────────────────────────────────────────────────┐
│ 0x0000 : RISC-V (RV64)                           │
│ 0x0001 : ARM64 (AArch64)                         │
│ 0x0002 : x86-64                                  │
│ 0x0003 : MIPS64                                  │
│ 0x0004 : PowerPC64                               │
│ 0xFF00-0xFFFF : Implementation-defined           │
└──────────────────────────────────────────────────┘
```

20.2 ISA Switch on Domain Transition

Domain transitions may involve ISA switch:

```
descend R0 (child_dcb):

Step 1: Read child ISA tag
──────────────────────────
child_isa = memory[R0 + isa_tag_offset]

Step 2: ISA validation
──────────────────────
if child_isa not in supported_isas:
    fault(UNSUPPORTED_ISA)

Step 3: ISA switch
──────────────────
current_isa = child_isa
```

(continues on next page)

(continued from previous page)

```
update_decoder(current_isa)

Step 4: Context format switch
─────────────────────────────
// Register file format may differ
// Saved context uses target ISA format
```

20.3 ISA Boundary Rules

ISA transitions follow specific rules:

Table 1: ISA Transition Rules

Transition	Behavior
Same ISA	Normal domain switch
Different ISA	ISA switch + context reformat
Unsupported ISA	Fault (UNSUPPORTED_ISA)

```
Example: RISC-V to ARM64 transition:

Domain 0 (RISC-V):
  └─ Domain 1 (ARM64):
     └─ Domain 2 (ARM64):

descend from Domain 0 to Domain 1:
- ISA switch: RISC-V → ARM64
- Register reformat: x0-x31 → X0-X30, SP, PC

descend from Domain 1 to Domain 2:
- Same ISA (ARM64)
- Normal domain switch
```

20.4 Binary Translation Integration

ISA tags enable binary translation:

```
Legacy application on modern ISA:

┌──────────────────────────────────────────────┐
│ Domain 0: ARM64 OS                           │
│   └─ Domain 1: x86-32 Application            │
│      (isa_tag = x86-32)                      │
│      (Binary translator in DCB)              │
└──────────────────────────────────────────────┘

Hardware can:
- Recognize ISA mismatch
```

(continues on next page)

(continued from previous page)

```
- Invoke binary translator
- Execute translated code
- Maintain x86-32 semantics
```

20.5 Implementation Requirements

Heterogeneous-ISA support requires:

```
Hardware:
- Multiple instruction decoders
- ISA-aware context save/restore
- ISA tag validation

Software:
- ISA-aware compilers
- Cross-ISA calling conventions
- Binary translation support (optional)
```

CHAPTER

TWENTYONE

CONTEXT MANAGEMENT

Context management in heterogeneous-ISA systems handles ISA-specific register formats.

21.1 Context Structure

Each ISA has a different context format:

```
RISC-V Context:
┌─────────────────────────────────────────────────┐
│ PC (64-bit)                                     │
│ x1-x31 (31 × 64-bit)                            │
│ F0-F31 (32 × 64-bit, if FP enabled)             │
│ CSR subset (sstatus, etc.)                      │
└─────────────────────────────────────────────────┘

ARM64 Context:
┌─────────────────────────────────────────────────┐
│ PC (64-bit)                                     │
│ SP (64-bit)                                     │
│ X0-X30 (31 × 64-bit)                            │
│ V0-V31 (32 × 128-bit, SIMD/FP)                  │
│ PSTATE subset                                   │
└─────────────────────────────────────────────────┘

x86-64 Context:
┌─────────────────────────────────────────────────┐
│ RIP (64-bit)                                    │
│ RSP (64-bit)                                    │
│ RFLAGS (64-bit)                                 │
│ RAX-R15 (16 × 64-bit)                           │
│ XMM0-XMM15 (16 × 128-bit, SSE)                  │
└─────────────────────────────────────────────────┘
```

21.2 Context Save on Domain Transition

Hardware saves context in ISA-specific format:

```
On ascend/descend:
```

(continues on next page)

(continued from previous page)

```
Step 1: Determine current ISA
─────────────────────────────
current_isa = current_dcb.isa_tag

Step 2: Save in ISA-specific format
───────────────────────────────────
switch current_isa:
    case RISCV:
        save_riscv_context(current_dcb)
    case ARM64:
        save_arm64_context(current_dcb)
    case X86_64:
        save_x86_64_context(current_dcb)
```

21.3 Context Restore

Context restore uses target ISA format:

```
On return:

Step 1: Determine target ISA
────────────────────────────
target_isa = child_dcb.isa_tag

Step 2: Restore in ISA-specific format
──────────────────────────────────────
switch target_isa:
    case RISCV:
        restore_riscv_context(child_dcb)
    case ARM64:
        restore_arm64_context(child_dcb)
    case X86_64:
        restore_x86_64_context(child_dcb)
```

21.4 Cross-ISA Calling Convention

Cross-ISA calls require argument mapping:

```
RISC-V → ARM64 argument mapping:

┌──────────┬──────┬──────────────────────────┐
│ RISC-V   │ ARM64 │ Purpose                  │
├──────────┼──────┼──────────────────────────┤
│ a0 (x10) │ X0    │ Argument 0 / Return value │
│ a1 (x11) │ X1    │ Argument 1               │
│ a2 (x12) │ X2    │ Argument 2               │
│ a3 (x13) │ X3    │ Argument 3               │
│ a4 (x14) │ X4    │ Argument 4               │
```

(continues on next page)

(continued from previous page)

```
| a5 (x15)| X5      | Argument 5                          |
| a6 (x16)| X6      | Argument 6                          |
| a7 (x17)| X7      | Argument 7                          |
└─────────┴─────────┴─────────────────────────────────────┘

Hardware performs this mapping automatically on
cross-ISA domain transitions.
```

21.5 Context Size

Context size varies by ISA:

Table 1: Context Sizes

ISA	Base Size	With Extensions
RISC-V RV64	264 bytes	+256 bytes (FP), + variable (V)
ARM64	528 bytes	+512 bytes (SVE)
x86-64	296 bytes	• variable (AVX-512)

```
DCB.ctrlblock_size must accommodate:
- Context for domain's ISA
- ISA-specific extensions
- Implementation-defined extensions
```

CHAPTER

TWENTYTWO

PRIVILEGE MODEL

RPA replaces traditional fixed privilege levels with flexible domain-based privilege management.

22.1 Traditional vs RPA

```
Traditional (e.g., ARM, x86, RISC-V):
┌─────────────────────────────────────────────────┐
│ Level 0: User Mode                              │
│ Level 1: Supervisor (OS)                        │
│ Level 2: Hypervisor                             │
│ Level 3: Secure Monitor                         │
│                                                 │
│ Levels are FIXED in hardware                    │
└─────────────────────────────────────────────────┘

RPA:
┌─────────────────────────────────────────────────┐
│ Domain N: Arbitrary privilege level             │
│ Domain N+1: Higher privilege                    │
│ Domain N+2: Even higher privilege               │
│                                                 │
│ Privilege determined by position in hierarchy   │
│ Number of levels is SOFTWARE-DEFINED            │
└─────────────────────────────────────────────────┘
```

22.2 Privilege Determination

A domain's privilege is relative to its position:

```
Relative privilege:

Domain A has HIGHER privilege than Domain B if:
- A is an ancestor of B in the domain tree
- A can intercept B's traps
- A can access B's memory (subject to security groups)

Domain A has EQUAL privilege to Domain B if:
- A and B are the same domain
```

(continues on next page)

(continued from previous page)

```
Domain A has LOWER privilege than Domain B if:
- A is a descendant of B in the domain tree
```

22.3 Privilege Inheritance

Child domains inherit restrictions from parents:

```
Child cannot:
- Access memory outside parent-granted IPA regions
- Use instructions parent has disabled
- Bypass parent's security policies

Parent can:
- Intercept any child trap
- Terminate child domain
- Modify child's DCB
```

22.4 Privilege Escalation

Privilege escalation requires explicit domain transition:

```
To gain privilege:
- ascend: Request service from higher-privilege domain
- Higher domain decides whether to grant access

Explicit escalation:
┌──────────────────────────────────────────────────┐
│ Application (low privilege):                     │
│   ascend SYSTEM_CALL                             │
│       │                                          │
│                                                  │
│ OS (higher privilege):                           │
│   - Validates request                            │
│   - Performs privileged operation                │
│   - return with result                           │
└──────────────────────────────────────────────────┘

No implicit escalation:
- Cannot "switch to kernel mode"
- Must go through domain boundary
- Parent always in control
```

22.5 Capability Model

RPA can implement capability-based security:

```
DCB as capability:

A DCB represents a capability to:
- Execute code
- Access specific memory regions
- Create child domains

Capability rules:
1. Can only create capabilities for resources you own
2. Cannot amplify your own capabilities
3. Can only reduce capabilities of children

Example:
┌─────────────────────────────────────────────────┐
│ Parent DCB: Full memory access                  │
│                                                 │
│ Child DCB creation:                             │
│   - Parent selects subset of memory             │
│   - Child gets restricted IPA regions           │
│   - Child cannot exceed parent's access         │
└─────────────────────────────────────────────────┘
```

CHAPTER

TWENTYTHREE

SECURITY OVERVIEW

RPA provides hardware-enforced security through domain isolation, access control, and memory protection.

23.1 Security Principles

RPA security is built on three principles:

Table 1: Security Principles

Principle	Description
Least Privilege	Domains have minimum necessary access
Isolation	Domains cannot access each other's memory
Delegation	Access can be delegated but not amplified

23.2 Threat Model

RPA addresses the following threats:

Table 2: Threats Addressed

Threat	Mitigation
Malicious application	IPA boundary enforcement
Compromised OS	Security groups prevent OS access
Malicious hypervisor	Nested isolation, confidential computing
Side-channel attacks	Memory encryption, timing isolation

23.3 Security Mechanisms

RPA provides multiple security mechanisms:

```
Mechanism          | Level        | Purpose
-------------------+--------------+-------------------------
IPA Boundaries     | Memory       | Region-level isolation
Page Permissions   | Page         | Fine-grained access control
Domain Hierarchy   | System       | Privilege separation
Security Groups    | Management   | Access control by group
```

(continues on next page)

(continued from previous page)

```
Memory Encryption    | Physical    | Memory confidentiality
Attestation          | Integrity   | Remote verification
```

23.4 Defense in Depth

RPA supports defense in depth:

```
Layer 1: Application Isolation
──────────────────────────────
Application cannot access other applications

Layer 2: OS Isolation
─────────────────────
OS cannot access security-group-protected memory

Layer 3: Hypervisor Isolation
─────────────────────────────
Hypervisor isolated from VMs via nested domains

Layer 4: Hardware Isolation
───────────────────────────
Memory encryption protects against physical attacks
```

23.5 Security Configuration

Security features are configurable per domain:

```
DCB Security Fields:
┌──────────────────────────────────────────────────┐
│ security_group    : Group membership             │
│ encryption_enabled : Memory encryption flag      │
│ attestation_key    : Attestation key reference   │
│ ipa_regions[]      : Memory access control       │
└──────────────────────────────────────────────────┘

Parent sets these fields on domain creation.
Child cannot modify its own security configuration.
```

CHAPTER

TWENTYFOUR

SECURITY GROUPS

Security groups provide access control independent of domain hierarchy, separating management rights from access rights.

24.1 Concept

Traditional domain hierarchy grants parent full access to child. Security groups restrict this access:

```
Without security groups:
┌──────────────────────────────────────────────────┐
│ Parent can:                                      │
│   - Access all child memory                      │
│   - Modify child's DCB                           │
│   - Intercept all child traps                    │
└──────────────────────────────────────────────────┘

With security groups:
┌──────────────────────────────────────────────────┐
│ Parent can:                                      │
│   - Manage child (create, terminate)             │
│   - BUT cannot access child's memory             │
│   - IF child is in different security group      │
└──────────────────────────────────────────────────┘
```

24.2 Security Group Definition

Each domain belongs to a security group:

```
DCB Field: security_group
┌──────────────────────────┐

Identifier of security group (0-255)

Security Group Table (system-wide):
┌──────────────────────────────────────────────────┐
│ Group 0: Root (full access)                      │
│ Group 1: Trusted OS                              │
│ Group 2: Applications                            │
│ Group 3: Secure Enclave                          │
```

(continues on next page)

(continued from previous page)

```
│ ...                                                    │
└────────────────────────────────────────────────────────┘
```

24.3 Access Control Matrix

Security groups define an access control matrix:

```
Access Matrix:
┌──────────┬─────────┬─────────┬─────────┬─────────┐
│          │ Group 0 │ Group 1 │ Group 2 │ Group 3 │
├──────────┼─────────┼─────────┼─────────┼─────────┤
│ Group 0  │   R/W   │   R/W   │   R/W   │   R/W   │
│ Group 1  │    -    │   R/W   │   R/W   │    -    │
│ Group 2  │    -    │    -    │   R/W   │    -    │
│ Group 3  │    -    │    -    │    -    │   R/W   │
└──────────┴─────────┴─────────┴─────────┴─────────┘

R/W = Can read and write
-   = No access
```

24.4 Memory Access Check

Hardware checks security group on memory access:

```
check_access(source_domain, target_memory):

source_group = source_domain.dcb.security_group
target_group = target_memory.owner_group

if access_matrix[source_group][target_group] == DENY:
    fault(SECURITY_GROUP_VIOLATION)
```

24.5 Example: Secure Enclave

```
Secure enclave protected from OS:

Domain 0: Root (Group 0)
  └─ Domain 1: OS (Group 1)
     ├─ Domain 2: Application (Group 2)
     └─ Domain 3: Secure Enclave (Group 3)

Access:
┌────────────────────────────────────────────────────────┐
│ OS → Application memory: ALLOWED                       │
│ OS → Secure Enclave memory: DENIED                     │
│ Application → Secure Enclave: DENIED                   │
│ Secure Enclave → Own memory: ALLOWED                   │
```

(continues on next page)

(continued from previous page)

```
└──────────────────────────────────────────────────┘

OS can still:
- Create/terminate enclave domain
- Allocate memory for enclave
- But cannot read enclave's private data
```

24.6 Management vs Access

Security groups separate management from access:

```
Management rights (always inherited from hierarchy):
- Create child domains
- Terminate child domains
- Configure child's DCB

Access rights (controlled by security groups):
- Read child's memory
- Write child's memory
- Execute child's code

This separation enables:
- Trusted execution environments
- Confidential computing
- Secure enclaves
```

CHAPTER

TWENTYFIVE

MEMORY ENCRYPTION

RPA supports hardware memory encryption to protect against physical memory attacks.

25.1 Encryption Overview

Memory encryption protects data at rest:

```
Memory Encryption Flow:
┌─────────────────────────────────────────────────────┐
│  CPU                                                │
│    │                                                │
│    │ Plain text                                     │
│    ▼                                                │
│  ┌──────────────────────────────────────────────┐   │
│  │ Encryption Engine                            │   │
│  │   key = domain_encryption_key                │   │
│  │   cipher = AES-XTS(plaintext, key)           │   │
│  └──────────────────────────────────────────────┘   │
│    │                                                │
│    │ Cipher text                                    │
│    ▼                                                │
│  ┌──────────────────────────────────────────────┐   │
│  │ Memory Controller                            │   │
│  └──────────────────────────────────────────────┘   │
│    │                                                │
│    ▼                                                │
│  DRAM (encrypted)                                   │
└─────────────────────────────────────────────────────┘
```

25.2 Per-Domain Encryption

Each domain can have its own encryption key:

```
DCB Field: encryption_key_id
─────────────────────────────

Reference to encryption key

Key Management:
─────────────────────────────────────────────────────
```

(continues on next page)

(continued from previous page)

```
┌──────────────────────────────────────────────────┐
│ Key 0: Root domain key                           │
│ Key 1: OS domain key                             │
│ Key 2: Application domain key                    │
│ Key 3: Secure enclave key                        │
└──────────────────────────────────────────────────┘

Hardware automatically:
1. Selects key based on accessing domain
2. Encrypts on write
3. Decrypts on read
```

25.3 Encryption Modes

RPA supports multiple encryption modes:

Table 1: Encryption Modes

Mode	Description
None	No encryption
Integrity-only	MAC but no encryption
Confidentiality	Encryption without integrity
Confidentiality+Integrity	Full encryption with MAC

```
DCB Field: encryption_mode
──────────────────────────

0 = None
1 = Integrity only
2 = Confidentiality
3 = Confidentiality + Integrity
```

25.4 Memory Bandwidth Impact

Encryption adds latency to memory access:

```
Latency components:
┌──────────────────────────────────────────────────┐
│ Base DRAM latency:    ~100 cycles                │
│ Encryption overhead:  +~10-20 cycles             │
│ Integrity check:      +~5-10 cycles              │
│ Total:                ~115-130 cycles            │
└──────────────────────────────────────────────────┘

Mitigations:
- Cache encryption keys in on-chip storage
- Use wide-bus encryption engines
- Cache line granularity encryption
```

25.5 Key Management

Encryption keys are managed securely:

```
Key Lifecycle:

1. Generation
   ──────────────
   Keys generated by hardware RNG
   Never exposed to software

2. Distribution
   ──────────────
   Keys stored in secure key storage
   Referenced by key_id in DCB

3. Rotation
   ──────────
   Parent can rotate child's key
   Requires domain re-encryption

4. Destruction
   ─────────────
   Key destroyed on domain termination
   Memory becomes unreadable
```

25.6 Use Case: Confidential Computing

```
Confidential VM protected from cloud hypervisor:

Domain 0: Cloud Hypervisor (Group 0, Key 0)
  └─ Domain 1: Confidential VM (Group 3, Key 1)

Cloud hypervisor:
- Can manage VM (create, terminate)
- Cannot read VM memory (different security group)
- Cannot decrypt VM memory (different key)

VM owner can verify:
- VM is isolated from hypervisor
- Memory is encrypted
- Via remote attestation
```

CHAPTER

TWENTYSIX

ATTESTATION

Attestation enables remote verification of domain integrity and isolation.

26.1 Attestation Overview

Attestation proves to a remote party:

```
Attestation proves:
┌─────────────────────────────────────────────────┐
│ 1. Domain is running expected code              │
│ 2. Domain is properly isolated                  │
│ 3. Domain has expected configuration            │
│ 4. Hardware security features are enabled       │
└─────────────────────────────────────────────────┘
```

26.2 Attestation Components

```
Attestation Report Structure:
┌─────────────────────────────────────────────────┐
│ Header                                          │
│   - Report version                              │
│   - Domain ID                                   │
│   - Timestamp                                   │
├─────────────────────────────────────────────────┤
│ Measurements                                    │
│   - Code hash (SHA-256)                         │
│   - Initial data hash                           │
│   - Configuration hash                          │
├─────────────────────────────────────────────────┤
│ Security State                                  │
│   - Security group                              │
│   - Encryption mode                             │
│   - IPA regions                                 │
│   - Parent domain ID                            │
├─────────────────────────────────────────────────┤
│ Platform State                                  │
│   - Hardware version                            │
│   - Firmware version                            │
```

(continues on next page)

(continued from previous page)

```
  - Security features enabled                          |
-------------------------------------------------------
Signature                                              |
  - Sign(report, attestation_key)                      |
-------------------------------------------------------
```

26.3 Attestation Keys

Each domain has an attestation key:

```
DCB Field: attestation_key_id
---------------------------------
Reference to attestation key

Key Hierarchy:
-------------------------------------------------------
Root Attestation Key (RAK)                             |
  - Provisioned by manufacturer                        |
  - Never leaves hardware                              |
  - Signs platform certificates                        |
-------------------------------------------------------
Domain Attestation Keys (DAK)                          |
  - Derived from RAK                                   |
  - Unique per domain                                  |
  - Signs domain attestation reports                   |
-------------------------------------------------------
```

26.4 Attestation Protocol

```
Remote Attestation Flow:

Verifier                Domain              Hardware
   |                      |                    |
   |  1. Challenge (nonce)|                    |
   |--------------------->|                    |
   |                      |                    |
   |                      | 2. Request report  |
   |                      |------------------->|
   |                      |                    |
   |                      | 3. Attestation     |
   |                      |    report          |
   |                      |<-------------------|
   |                      |                    |
   |  4. Attestation report                    |
   |<---------------------|                    |
   |                      |                    |
   |  5. Verify signature |                    |
   |  6. Check measurements                    |
```

(continues on next page)

(continued from previous page)

```
   │  7. Trust decision      │                        │
   │                         │                        │
```

26.5 Measurement Chain

Measurements form a chain of trust:

```
Measurement Chain:
┌─────────────────────────────────────────────────┐
│ M0: Boot firmware                               │
│ M1: Hypervisor (hash of M0 + hypervisor code)   │
│ M2: OS (hash of M1 + OS code)                   │
│ M3: Application (hash of M2 + app code)         │
└─────────────────────────────────────────────────┘

Each measurement includes:
- Hash of code
- Hash of parent measurement
- Configuration at creation time

Verification:
- Verifier knows expected M3
- Can compute expected M2, M1, M0
- Compares with attestation report
```

CHAPTER

TWENTYSEVEN

CONFIDENTIAL COMPUTING

RPA enables confidential computing through domain isolation and hardware-enforced security.

27.1 Confidential Computing Model

Confidential computing protects data in use:

```
Traditional Protection:
┌─────────────────────────────────────────────┐
│ Data at rest:    Encryption ✓               │
│ Data in transit: Encryption ✓               │
│ Data in use:     Plaintext                  │
└─────────────────────────────────────────────┘

Confidential Computing:
┌─────────────────────────────────────────────┐
│ Data at rest:    Encryption ✓               │
│ Data in transit: Encryption ✓               │
│ Data in use:     Isolated ✓                 │
└─────────────────────────────────────────────┘
```

27.2 Threat Model

Confidential computing protects against privileged attackers:

Table 1: Protected Against

Attacker	Mitigation
Malicious admin	Domain isolation, security groups
Compromised OS	Memory encryption, IPA boundaries
Compromised hypervisor	Nested isolation, attestation
Physical attacker	Memory encryption, secure key storage

27.3 RPA Confidential Computing Architecture

```
Confidential VM Architecture:
```

(continues on next page)

(continued from previous page)

```
┌─────────────────────────────────────────────────────┐
│ Cloud Provider (Untrusted)                          │
│ ┌─────────────────────────────────────────────────┐ │
│ │ Domain 0: Cloud Hypervisor                      │ │
│ │   - Manages VM lifecycle                        │ │
│ │   - Cannot access VM memory                     │ │
│ │   - Cannot read VM secrets                      │ │
│ │ ┌─────────────────────────────────────────────┐ │ │
│ │ │ Domain 1: Confidential VM                   │ │ │
│ │ │   - Security group: Isolated                │ │ │
│ │ │   - Memory: Encrypted                       │ │ │
│ │ │   - Attestation: Enabled                    │ │ │
│ │ │                                             │ │ │
│ │ │   - Customer workload                       │ │ │
│ │ │   - Customer data                           │ │ │
│ │ │   - Customer keys                           │ │ │
│ │ └─────────────────────────────────────────────┘ │ │
│ └─────────────────────────────────────────────────┘ │
└─────────────────────────────────────────────────────┘
```

27.4 Security Configuration

Confidential VM DCB configuration:

```
Confidential VM DCB:
┌─────────────────────────────────────────────────────┐
│ security_group    = 3  (Isolated)                   │
│ encryption_mode   = 3  (Confidentiality+Integrity)  │
│ encryption_key_id = 1  (Unique to VM)               │
│ attestation_key_id= 1  (Unique to VM)               │
│ ipa_regions[]     = VM's assigned memory            │
└─────────────────────────────────────────────────────┘

Effect:
- Hypervisor cannot read VM memory (security group)
- VM memory encrypted with unique key
- VM can attest its configuration
```

27.5 VM Launch Protocol

```
Confidential VM Launch:

1. Image Loading
──────────────────
- Hypervisor loads VM image
- Image is encrypted (customer key)
- Hypervisor cannot read plaintext
```

(continues on next page)

(continued from previous page)

```
2. Domain Creation
────────────────────
- Hypervisor creates domain
- Sets security_group = Isolated
- Sets encryption_key_id
- Sets attestation_key_id

3. Secret Injection
────────────────────
- Customer sends secrets
- Encrypted with VM's public key
- Only VM can decrypt

4. Attestation
──────────────
- VM generates attestation report
- Customer verifies report
- Customer trusts VM
```

27.6 Use Cases

Table 2: Confidential Computing Use Cases

Use Case	Description
Cloud ML training	Protect training data from cloud provider
Financial processing	Protect transaction data
Healthcare	Process patient data in compliance
Blockchain	Secure key management for nodes

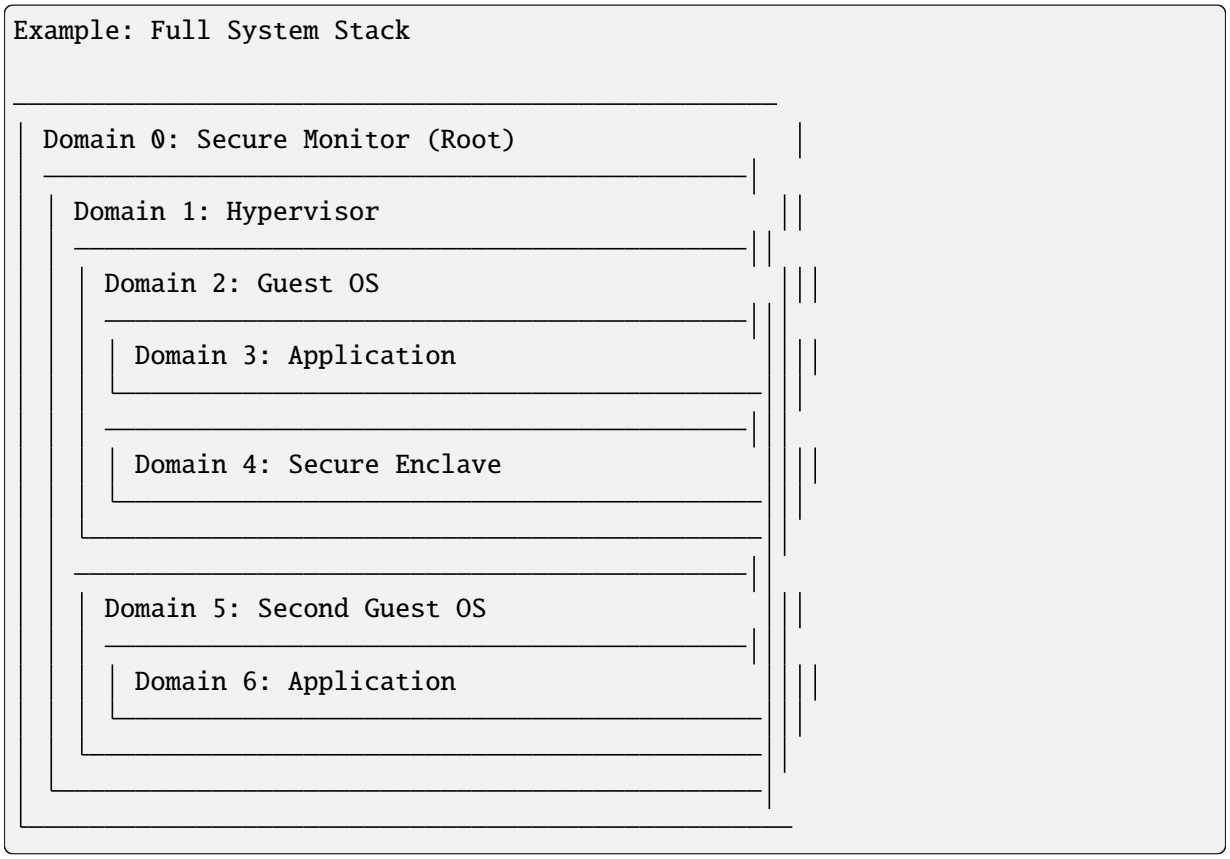
CHAPTER

TWENTYEIGHT

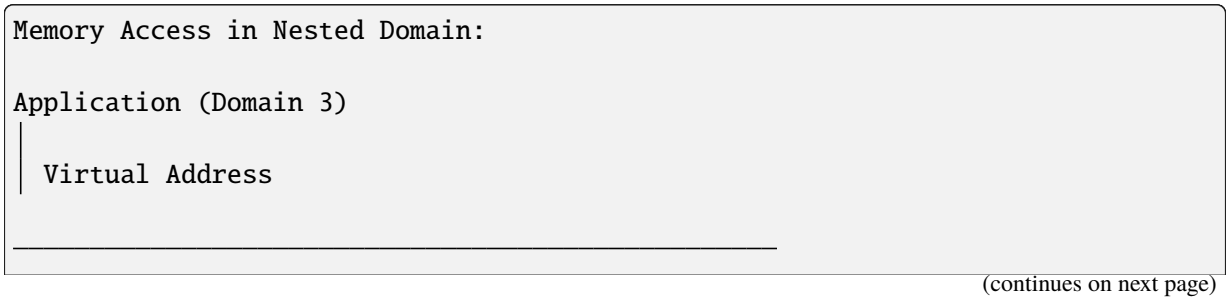
SYSTEM DIAGRAM

This chapter provides visual overview of RPA system organization.

28.1 Domain Hierarchy



28.2 Address Translation Flow



(continues on next page)

(continued from previous page)

```
┌──────────────────────────────────────────────────┐
│ Domain 3 Page Table                              │
│ VA → IPA (Guest Physical)                        │
└──────────────────────────────────────────────────┘
                       │
                       │ Intermediate Physical Address
                       ▼
┌──────────────────────────────────────────────────┐
│ IPA Boundary Check                               │
│ Is IPA within Domain 3's permitted regions?      │
└──────────────────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────┐
│ Domain 2 (Guest OS) Page Table                   │
│ IPA → IPA (Hypervisor Physical)                  │
└──────────────────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────┐
│ Domain 1 (Hypervisor) Page Table                 │
│ IPA → PA (Physical)                              │
└──────────────────────────────────────────────────┘
                       │
                       │ Physical Address
                       ▼
┌──────────────────────────────────────────────────┐
│ Memory                                           │
└──────────────────────────────────────────────────┘
```

28.3 Domain Transition

```
ascend Primitive:

Domain 3 (Application)          Domain 2 (OS)
┌──────────────────────┐       ┌──────────────────────┐
│ Executing            │       │                      │
│                      │       │                      │
│ ascend SYSCALL       │       │                      │
│ ─────────────────────────────>│                      │
│                      │       │ Trap handler         │
│                      │       │ executes             │
│                      │       │                      │
│                      │       │ return               │
│ <─────────────────────────────│                      │
│ Resumes execution    │       │                      │
└──────────────────────┘       └──────────────────────┘

Timeline:
```

(continues on next page)

(continued from previous page)

```
1. Application executes normally
2. Application executes ascend instruction
3. Hardware saves context to Domain 3 DCB
4. Hardware switches to Domain 2
5. Domain 2 trap handler executes
6. Domain 2 executes return instruction
7. Hardware restores Domain 3 context
8. Application resumes execution
```

28.4 Security Group Isolation

```
Security Group Access Control:

┌─────────────────────────────────────────────────┐
│ Domain 1: OS (Group 1)                          │
│ ┌─────────────────────────────────────────────┐ │
│ │ Domain 2: App A (Group 2)                   │ │
│ │ [Memory: 0x10000-0x1FFFF]                   │ │
│ └─────────────────────────────────────────────┘ │
│ ┌─────────────────────────────────────────────┐ │
│ │ Domain 3: App B (Group 2)                   │ │
│ │ [Memory: 0x20000-0x2FFFF]                   │ │
│ └─────────────────────────────────────────────┘ │
│ ┌─────────────────────────────────────────────┐ │
│ │ Domain 4: Enclave (Group 3)                 │ │
│ │ [Memory: 0x30000-0x3FFFF]                   │ │
│ │ [Encrypted with Key 1]                      │ │
│ └─────────────────────────────────────────────┘ │
└─────────────────────────────────────────────────┘

Access Matrix:
┌──────────┬─────────┬─────────┬─────────┬─────────┐
│ From/To  │ App A   │ App B   │ Enclave │ OS      │
├──────────┼─────────┼─────────┼─────────┼─────────┤
│ OS       │  R/W    │  R/W    │   -     │  R/W    │
│ App A    │  R/W    │   -     │   -     │   -     │
│ App B    │   -     │  R/W    │   -     │   -     │
│ Enclave  │   -     │   -     │  R/W    │   -     │
└──────────┴─────────┴─────────┴─────────┴─────────┘
```

CHAPTER

TWENTYNINE

HARDWARE-SOFTWARE INTERFACE

This chapter describes the interface between RPA hardware and software.

29.1 DCB Memory Layout

The Domain Control Block is the primary hardware-software interface:

```
DCB Memory Layout:
┌──────────────┬──────────────────────────────────────┐
│ Offset       │ Field                                │
├──────────────┼──────────────────────────────────────┤
│ 0x00         │ ctrlblock_size (32-bit)              │
│ 0x04         │ domain_id (32-bit)                   │
│ 0x08         │ parent_block (64-bit)                │
│ 0x10         │ child_block (64-bit)                 │
│ 0x18         │ trap_vector (64-bit)                 │
│ 0x20         │ interrupt_vector (64-bit)            │
│ 0x28         │ page_table_root (64-bit)             │
│ 0x30         │ translation_mode (32-bit)            │
│ 0x34         │ isa_tag (16-bit)                     │
│ 0x36         │ security_group (8-bit)               │
│ 0x37         │ encryption_mode (8-bit)              │
│ 0x38         │ saved_pc (64-bit)                    │
│ 0x40         │ saved_sp (64-bit)                    │
│ 0x48         │ saved_psr (64-bit)                   │
│ 0x50+        │ ISA-specific registers               │
│ ...          │ ...                                  │
│ variable     │ ipa_regions[]                        │
└──────────────┴──────────────────────────────────────┘
```

29.2 Instruction Encoding

RPA primitives use standard instruction encoding:

```
descend encoding:
┌──────────────────────────────────────────────────────┐
│ [31:26] opcode = 0x3F (CUSTOM)                       │
│ [25:20] func = 0x00 (DESCEND)                        │
│ [19:15] rs1 = DCB address register                   │
```

(continues on next page)

(continued from previous page)

```
  [14:0]  unused                                          

ascend encoding:
                                                          
  [31:26] opcode = 0x3F (CUSTOM)                          
  [25:20] func = 0x01 (ASCEND)                            
  [19:15] rs1 = service type register                     
  [14:0]  unused                                          

return encoding:
                                                          
  [31:26] opcode = 0x3F (CUSTOM)                          
  [25:20] func = 0x02 (RETURN)                            
  [19:0]  unused                                          
```

29.3 Control Registers

RPA defines system control registers:

```
Control Registers:
                                                          
  Address    Name            Description                  
                                                          
  0x800      CURRENT_DCB     Current domain's DCB         
  0x804      DOMAIN_ID       Current domain ID            
  0x808      PARENT_DCB      Parent's DCB                 
  0x80C      CHILD_DCB       Child's DCB                  
  0x810      TRAP_STATUS     Trap status register         
  0x814      FAULT_CODE      Fault code register          

Access:
- Read-only: DOMAIN_ID, CURRENT_DCB
- Read-write: TRAP_STATUS (by handler)
- Write-only: FAULT_CODE (hardware writes)
```

29.4 Software Conventions

Software should follow these conventions:

```
DCB Allocation:
- DCB must be 32-byte aligned
- DCB size must be multiple of 32 bytes
- DCB should be in memory accessible to parent
```

(continues on next page)

(continued from previous page)

```
Register Usage:
- R0 (or X0/a0): Argument/return value
- R1 (or X1/a1): Second argument
- PC: Saved on domain transition
- SP: Saved on domain transition

Trap Handler Convention:
┌──────────────────────────────────────────────────┐
│ Entry:                                           │
│   R0 = trap_type                                 │
│   R1 = faulting_address                          │
│   SP = handler stack (configured)                │
│                                                  │
│ Exit:                                            │
│   return instruction                             │
│   R0 = return value                              │
└──────────────────────────────────────────────────┘
```

29.5 Error Codes

Hardware reports errors via fault codes:

Table 1: Fault Codes

Code	Description
0x00	No error
0x01	DCB alignment error
0x02	DCB size error
0x03	No child domain
0x04	Child not suspended
0x10	IPA boundary violation
0x11	Permission denied
0x12	Security group violation
0x20	Unhandled trap
0x21	Unhandled interrupt
0x30	Unsupported ISA

CHAPTER

THIRTY

BOOT SEQUENCE

This chapter describes the RPA system boot sequence.

30.1 Boot Stages

```
Boot Sequence:

Stage 0: Hardware Reset
─────────────────────────
- Hardware initializes
- Creates root domain (Domain 0)
- Loads initial DCB at fixed address

Stage 1: Boot Firmware
──────────────────────
- Firmware executes in Domain 0
- Initializes memory, devices
- Verifies next stage (optional)
- Creates Domain 1 for bootloader/OS

Stage 2: Bootloader/Hypervisor
──────────────────────────────
- Executes in Domain 1
- Loads OS/hypervisor
- Creates Domain 2 for OS

Stage 3: Operating System
─────────────────────────
- Executes in Domain 2
- Initializes kernel
- Creates Domain 3+ for applications
```

30.2 Initial DCB Configuration

```
Hardware creates initial DCB:

Root DCB (created by hardware):
──────────────────────────────────────────────────
```

(continues on next page)

(continued from previous page)

```
ctrlblock_size    = minimum_size
domain_id         = 0
parent_block      = 0 (no parent)
child_block       = 0 (no child yet)
trap_vector       = 0 (must be set by firmware)
interrupt_vector  = 0
page_table_root   = identity_mapping
translation_mode  = OWN
isa_tag           = system_isa
security_group    = 0 (root)
saved_pc          = reset_vector
saved_sp          = 0
ipa_regions[]     = full_physical_memory
```

30.3 Domain Creation During Boot

```
Firmware creates bootloader domain:

Step 1: Allocate DCB
____________________
dcb_addr = allocate_memory(DCB_SIZE)
assert(dcb_addr % 32 == 0)  // Must be aligned

Step 2: Initialize DCB
______________________
memory[dcb_addr + ctrlblock_size_offset] = DCB_SIZE
memory[dcb_addr + domain_id_offset] = 0  // Hardware assigns
memory[dcb_addr + parent_block_offset] = root_dcb
memory[dcb_addr + trap_vector_offset] = trap_handler_addr
memory[dcb_addr + page_table_root_offset] = page_table_addr
memory[dcb_addr + isa_tag_offset] = RISCV
memory[dcb_addr + security_group_offset] = 0
memory[dcb_addr + saved_pc_offset] = bootloader_entry

Step 3: Configure IPA Regions
_____________________________
memory[dcb_addr + ipa_regions_offset + 0] = {
    base:  0x80000000,
    limit: 0x80FFFFFF,
    attr:  RWX
}

Step 4: Descend to New Domain
_____________________________
R0 = dcb_addr
descend R0
```

30.4 Boot Example: Full System

```
Timeline:

T0: Hardware reset
    - Create Domain 0 (root)
    - PC = 0xFFFFFFF0 (reset vector)

T1: Firmware initialization
    - Initialize memory controller
    - Initialize essential devices
    - Load bootloader

T2: Create Domain 1 (bootloader)
    - Allocate DCB for bootloader
    - Configure IPA regions
    - descend to bootloader

T3: Bootloader execution
    - Load hypervisor/OS
    - Create Domain 2 (hypervisor)
    - descend to hypervisor

T4: Hypervisor initialization
    - Initialize virtualization
    - Create Domain 3 (guest OS)
    - descend to guest OS

T5: Guest OS initialization
    - Initialize kernel
    - Start services
    - Create Domain 4+ (applications)
```

30.5 Secure Boot

RPA supports secure boot through measurement:

```
Secure Boot Flow:

1. Hardware verifies firmware signature
   - Using ROM public key
   - Firmware hash stored in TPM

2. Firmware verifies bootloader
   - Using firmware's public key
   - Bootloader hash extends measurement

3. Bootloader verifies OS
   - Using bootloader's public key
   - OS hash extends measurement
```

(continues on next page)

(continued from previous page)

```
4. Measurement chain available for attestation
   - Full chain of trust
   - Remote verification possible

DCB fields for secure boot:
┌──────────────────────────────────────────────────┐
│ Each domain's DCB contains:                      │
│   - Code measurement (hash)                      │
│   - Configuration measurement                    │
│   - Parent measurement reference                 │
└──────────────────────────────────────────────────┘
```

CHAPTER

THIRTYONE

DCB FIELD ENCODING

This appendix provides detailed encoding for DCB fields.

31.1 Field Offsets

DCB Field Offsets (bytes):

Offset	Field
0x00	ctrlblock_size
0x04	domain_id
0x08	parent_block
0x10	child_block
0x18	trap_vector
0x20	interrupt_vector
0x28	interrupt_mask
0x30	page_table_root
0x38	translation_control
0x40	saved_pc
0x48	saved_sp
0x50	saved_psr
0x58	saved_lr
0x60	isa_tag
0x62	security_group
0x63	encryption_mode
0x64	encryption_key_id
0x68	attestation_key_id
0x70	ipa_region_count
0x78	ipa_regions[0].base
0x80	ipa_regions[0].limit
0x88	ipa_regions[0].attributes
...	(subsequent regions)

31.2 Field Descriptions

31.2.1 ctrlblock_size (Offset 0x00)

```
Size: 32 bits
Description: Size of DCB in bytes
Constraints:
- Must be  8 (minimum header size)
- Must be multiple of 32 (alignment)
Usage: Hardware validates on descend
```

31.2.2 domain_id (Offset 0x04)

```
Size: 32 bits
Description: Unique domain identifier
Constraints:
- Assigned by hardware on descend
- Read-only after assignment
Usage: Identifies domain in TLB, attestation
```

31.2.3 parent_block (Offset 0x08)

```
Size: 64 bits
Description: Physical address of parent's DCB
Constraints:
- Set by parent on domain creation
- Must be valid DCB address
Usage: Hardware traverses to parent on ascend
```

31.2.4 child_block (Offset 0x10)

```
Size: 64 bits
Description: Physical address of child's DCB
Constraints:
- 0 if no active child
- Set by parent on descend
Usage: Hardware uses on return
```

31.2.5 trap_vector (Offset 0x18)

```
Size: 64 bits
Description: Entry point for trap handler
Constraints:
- 0 = no handler (propagate to parent)
- Must be in accessible IPA region
Usage: Hardware jumps here on trap
```

31.3 IPA Region Attributes

ipa_regions[].attributes encoding:

Bits	Field
0	Readable (R)
1	Writable (W)
2	Executable (X)
3	Device memory (D)
4	Cacheable (C)
5	Bufferable (B)
6-7	Reserved
8-15	Memory type extension

31.4 Translation Control

translation_control encoding:

Bits	Field
0	Translation enable 0 = INHERIT (use parent's tables) 1 = OWN (use own page tables)
1-2	Page table format 0 = RISC-V Sv39 1 = RISC-V Sv48 2 = ARM64 3 = x86-64
3-7	Reserved

CHAPTER

THIRTYTWO

PSEUDOCODE

This appendix provides pseudocode for RPA primitive implementations.

32.1 Reference Implementation

A Python executable specification (pysim) validates the RPA design [PYSIM]. The pseudocode examples in this chapter are derived from the pysim implementation.

Repository: https://github.com/arthurdev000/rpa-pysim

Version: 0.7.0

Test Coverage: 107 test cases

32.2 descend Primitive

```
procedure descend(dcb_address: physical_address)

// Step 1: DCB Validation
size = memory[dcb_address + 0x00]
if dcb_address % 32 != 0:
    fault(DCB_ALIGNMENT_ERROR)

if size < 8:
    fault(DCB_SIZE_ERROR)

// Step 2: Domain ID Assignment
domain_id = next_domain_id
next_domain_id = next_domain_id + 1
memory[dcb_address + 0x04] = domain_id

// Step 3: Save Parent Context
current_dcb.saved_pc = PC + 4
current_dcb.saved_sp = SP
current_dcb.saved_psr = PSR

// Step 4: Link Child
current_dcb.child_block = dcb_address
memory[dcb_address + 0x08] = current_dcb_physical_address
```

(continues on next page)

(continued from previous page)

```
// Step 5: Domain Switch
current_domain = domain_id
current_dcb_physical_address = dcb_address

// Step 6: Load Child Entry
entry_point = memory[dcb_address + saved_lr_offset]
PC = entry_point

end procedure
```

32.3 ascend Primitive

```
procedure ascend(service_type: integer)

// Step 1: Context Save
current_dcb.saved_pc = PC + 4
current_dcb.saved_sp = SP
current_dcb.saved_psr = PSR

// Step 2: Get Parent DCB
parent_dcb_address = current_dcb.parent_block
if parent_dcb_address == 0:
    fault(UNHANDLED_TRAP)
    return

// Step 3: Trap Vector Resolution with Propagation
trap_handler = 0
while trap_handler == 0:
    trap_handler = memory[parent_dcb_address + trap_vector_offset]

    if trap_handler != 0:
        break

    parent_dcb_address = memory[parent_dcb_address + parent_block_offset]
    if parent_dcb_address == 0:
        fault(UNHANDLED_TRAP)
        return

// Step 4: Domain Switch
current_dcb_physical_address = parent_dcb_address
current_domain = memory[parent_dcb_address + domain_id_offset]

// Step 5: Jump to Handler
PC = trap_handler

// Pass service type in argument register
R0 = service_type

end procedure
```

32.4 return Primitive

```
procedure return()

// Step 1: Locate Child DCB
child_dcb_address = current_dcb.child_block
if child_dcb_address == 0:
    fault(NO_CHILD_DOMAIN)
    return

// Step 2: Domain Switch
current_dcb_physical_address = child_dcb_address
current_domain = memory[child_dcb_address + domain_id_offset]

// Step 3: Context Restore
PC = memory[child_dcb_address + saved_pc_offset]
SP = memory[child_dcb_address + saved_sp_offset]
PSR = memory[child_dcb_address + saved_psr_offset]

// Return value in R0
// (Handler sets R0 before return)

// Step 4: Clear Child Link
current_dcb.child_block = 0

end procedure
```

32.5 IPA Boundary Check

```
function check_ipa_access(ipa: address, access_type: access_type)
                          returns boolean

region_count = current_dcb.ipa_region_count

for i = 0 to region_count - 1:
    region_base = current_dcb.ipa_regions[i].base
    region_limit = current_dcb.ipa_regions[i].limit
    region_attr = current_dcb.ipa_regions[i].attributes

    if ipa >= region_base and ipa <= region_limit:
        // Found matching region, check permissions
        if access_type == READ and region_attr.R == 0:
            return PERMISSION_DENIED
        if access_type == WRITE and region_attr.W == 0:
            return PERMISSION_DENIED
        if access_type == EXECUTE and region_attr.X == 0:
            return PERMISSION_DENIED

        return ALLOWED
```

(continues on next page)

(continued from previous page)

```
// IPA not in any permitted region
return IPA_BOUNDARY_VIOLATION

end function
```

32.6 Security Group Check

```
function check_security_group_access(source_domain: domain_id,
                                     target_address: address)
                                     returns boolean

source_group = domain_table[source_domain].security_group
target_group = get_memory_owner_group(target_address)

// Check access matrix
if access_matrix[source_group][target_group] == DENY:
    return SECURITY_GROUP_VIOLATION

return ALLOWED

end function
```

CHAPTER

THIRTYTHREE

CUSTOM EXTENSIONS

RPA is designed to be extensible. This appendix describes how to add custom extensions.

33.1 Extension Points

RPA provides several extension points:

Table 1: Extension Points

Extension Point	Description
DCB fields	Add implementation-defined fields
ISA tags	Support custom ISAs
Trap types	Define custom trap reasons
Memory attributes	Custom memory types

33.2 Custom DCB Fields

```
Standard DCB fields end at offset defined by ctrlblock_size.
Implementation-defined fields follow:

┌─────────────────────────────────────────────────┐
│ Standard DCB fields (0x00 - 0x6F)               │
├─────────────────────────────────────────────────┤
│ IPA regions (variable)                          │
├─────────────────────────────────────────────────┤
│ Implementation-defined fields                   │
│   0xN+00: custom_field_0                        │
│   0xN+08: custom_field_1                        │
│   ...                                           │
└─────────────────────────────────────────────────┘

Guidelines:
- ctrlblock_size must include custom fields
- Custom fields must be documented
- Software must check ctrlblock_size before accessing
```

33.3 Custom ISA Tags

```
ISA tag allocation:
┌──────────────────────────────────────────────────┐
│ 0x0000 - 0x00FF : Standard ISAs (RISC-V, ARM, x86)│
│ 0x0100 - 0xFEFF : Reserved for future standard   │
│ 0xFF00 - 0xFFFF : Implementation-defined         │
└──────────────────────────────────────────────────┘

Custom ISA implementation:
1. Define ISA tag value (0xFF00+)
2. Implement instruction decoder
3. Define context save/restore format
4. Document in implementation manual
```

33.4 Custom Trap Types

```
Trap type allocation:
┌──────────────────────────────────────────────────┐
│ 0x00 - 0x7F : Standard traps                     │
│ 0x80 - 0xFF : Implementation-defined             │
└──────────────────────────────────────────────────┘

Standard trap types:
┌──────────────────────────────────────────────────┐
│ 0x00 : System call                               │
│ 0x01 : Page fault                                │
│ 0x02 : Privilege violation                       │
│ 0x03 : IPA boundary violation                    │
│ 0x04 : Alignment fault                           │
│ 0x05 : Security group violation                  │
└──────────────────────────────────────────────────┘

Custom trap handling:
1. Hardware detects custom condition
2. Hardware sets trap_type = 0x80+
3. Standard trap propagation applies
4. Handler interprets custom trap_type
```

33.5 Custom Memory Attributes

```
Memory attribute extension:
┌──────────────────────────────────────────────────┐
│ Standard attributes (bits 0-7)                   │
│   R, W, X, D, C, B                               │
├──────────────────────────────────────────────────┤
│ Extension attributes (bits 8-15)                 │
│   Implementation-defined                         │
```

(continues on next page)

(continued from previous page)

```
└──────────────────────────────────────────────────┘

Example custom attributes:
┌──────────────────────────────────────────────────┐
│ Bit 8  : Scratchpad memory                       │
│ Bit 9  : Tightly-coupled memory                  │
│ Bit 10 : Non-coherent                            │
└──────────────────────────────────────────────────┘
```

33.6 Extension Discovery

Software can discover available extensions:

```
Extension Discovery Register:
┌──────────────────────────────────────────────────┐
│ Address: 0xF00                                   │
│ Bits 0-7  : Supported standard features          │
│ Bits 8-15 : Supported custom extensions          │
│ Bits 16-23: Extension version                    │
│ Bits 24-31: Implementation ID                    │
└──────────────────────────────────────────────────┘

Software should:
1. Read extension discovery register
2. Check required features are present
3. Adapt behavior based on available extensions
```

CHAPTER

THIRTYFOUR

REFERENCES

34.1 Related Architectures

Table 1: Related Work

Architecture	Relationship
ARM v8-A	Exception levels (EL0-EL3), virtualization
RISC-V	Privilege modes, physical memory protection
x86-64	Protection rings, VMX
Intel SGX	Enclave isolation, attestation
AMD SEV	Encrypted VMs, secure nested paging
ARM CCA	Realm domains, granule protection

34.2 Normative References

34.3 Informative References

34.4 Glossary

DCB
: Domain Control Block. Memory-resident data structure containing domain configuration.

IPA
: Intermediate Physical Address. Address after first-level translation in nested systems.

INHERIT
: Translation mode where child uses parent's page tables.

ascend
: RPA primitive for returning to parent domain.

descend
: RPA primitive for entering a child domain.

return
: RPA primitive for resuming child domain execution.

Security Group
: Access control mechanism separating management rights from access rights.

Domain
: Isolated execution context with configurable privilege.

CHAPTER

THIRTYFIVE

LEGAL NOTICE

35.1 Copyright

This specification is copyrighted by the Shanghai HappyFull Technologies.

```
SPDX-FileCopyrightText: Copyright 2026 Shanghai HappyFull Technologies
```

35.2 License

This specification is licensed under Creative Commons Attribution-ShareAlike 4.0 International (CC-BY-SA-4.0).

```
SPDX-License-Identifier: CC-BY-SA-4.0

You are free to:
- Share: copy and redistribute the material
- Adapt: remix, transform, and build upon the material

Under the following terms:
- Attribution: You must give appropriate credit
- ShareAlike: If you remix/transform, you must distribute
              under the same license
```

35.3 Trademarks

RPA and related trademarks are the property of the Shanghai HappyFull Technologies.

35.4 Disclaimers

This specification is provided “as is” without warranty of any kind.

The Shanghai HappyFull Technologies makes no representations about the suitability of this specification for any purpose.

35.5 Implementations

Implementations of this specification may be subject to additional licensing requirements from patent holders.

The Shanghai HappyFull Technologies does not guarantee that implementation of this specification does not infringe any patents.

35.6 Contact

For licensing inquiries or contributions:

```
Shanghai HappyFull Technologies
https://github.com/rpa-architecture/rpa-spec
```

CHAPTER

THIRTYSIX

INDICES AND TABLES

- genindex
- search

BIBLIOGRAPHY

[ARMARM] ARM Architecture Reference Manual ARMv8, for ARMv8-A architecture profile, ARM Ltd.

[RISCVPRIV] The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, RISC-V Foundation.

[INTELSDM] Intel 64 and IA-32 Architectures Software Developer's Manual, Intel Corporation.

[AMDAPM] AMD64 Architecture Programmer's Manual, AMD Inc.

[PYSIM] RPA Python Simulator (pysim), https://github.com/arthurdev000/rpa-pysim, Version 0.7.0, 107 test cases.

[SGX] Intel SGX Explained, Costan and Devadas, IACR Cryptology ePrint Archive, 2016.

[SEV] SEV-ES Architecture Overview, AMD Inc., 2020.

[CCA] Arm Confidential Compute Architecture, ARM Ltd., 2021.

[PMP] Physical Memory Protection in RISC-V, RISC-V Foundation.

[NESTED] Performance Analysis of Nested Virtualization, Armbrust et al., USENIX ATC, 2022.